

Selenium Interview Questions & Answers

Complete Guide — 160 Questions with Detailed Answers (InterviewBit + GeeksforGeeks)

Q1.

[Freshers]

What is Selenium?

Selenium is a free, open-source automated testing framework for validating web applications across multiple browsers and platforms. Test scripts can be written in Java, C#, Python, Ruby, and more.

Selenium is a suite of tools:

- Selenium IDE – Browser plugin that records and replays user actions
- Selenium RC – Server-based tool (now largely deprecated)
- Selenium WebDriver – Directly communicates with browsers using native support
- Selenium Grid – Runs tests in parallel across multiple browsers/machines

Q2.

[Freshers]

What are the Selenium suite components?

Selenium IDE: A Firefox/Chrome plugin for recording and replaying test scripts. Good for beginners.

Selenium RC (Remote Control): A server that accepts commands and sends them to the browser via JavaScript. Now deprecated.

Selenium WebDriver: Directly interacts with the browser without a server. Supports multiple languages and browsers.

Selenium Grid: Allows running tests in parallel on different machines and browsers simultaneously.

Q3.

[Freshers]

What are the advantages of using Selenium as an automation tool?

- Free and open-source
- Supports multiple programming languages (Java, Python, C#, Ruby, JavaScript, etc.)
- Compatible with many browsers (Chrome, Firefox, IE, Safari, Edge)
- Cross-platform (Windows, Mac, Linux)
- Large community and rich ecosystem
- Integrates with frameworks like TestNG, JUnit, Maven, Jenkins
- Supports parallel test execution via Selenium Grid
- Flexible locator strategies (ID, XPath, CSS, etc.)

Q4.

[Freshers]

What are the disadvantages of using Selenium as a testing tool?

- Supports only web-based applications (no desktop/mobile native apps)
- No built-in reporting; needs third-party tools

- Slow test execution compared to some commercial tools
- Cannot handle Windows-based pop-ups natively
- Requires programming knowledge
- No built-in image comparison
- Limited support for handling CAPTCHAs
- Maintenance overhead for dynamic applications

Q5.

[Freshers]

What is automation testing, and what are its advantages?

Automation testing is the process of using software tools to execute pre-scripted tests on an application, comparing actual outcomes with expected outcomes automatically.

Advantages:

- Faster execution than manual testing
- Reusable test scripts
- Improved accuracy and consistency
- Supports regression testing
- Better ROI for large projects
- Can run 24/7 without human intervention
- Enables parallel and load testing

Q6.

[Freshers]

Why should Selenium be selected as a testing tool for web applications?

- Open-source and free
- Supports all major browsers and OS platforms
- Multi-language support (Java, Python, C#, Ruby, etc.)
- Strong community support
- Easy integration with CI/CD tools (Jenkins, GitHub Actions)
- Supports Agile and DevOps practices
- Extensive documentation and tutorials
- Compatible with frameworks like TestNG, JUnit, Cucumber

Q7.

[Freshers]

What is Selenese? Explain different types of Selenium commands.

Selenese is the set of commands used in Selenium IDE to write test scripts.

Types of Selenium commands:

1. Actions – Directly interact with the application (click, type, select, dragAndDrop, etc.)
2. Accessors – Read values from the web application and store them in variables (storeTitle, storeText, etc.)
3. Assertions – Verify that the application is in the expected state:

- assert – Stops the test if the condition fails
- verify – Logs a failure but continues execution
- waitFor – Waits until a condition is true

Q8.

[Freshers]

What is a locator and name different types of locators in Selenium?

A locator is used to identify a web element uniquely on a web page.

Types of locators:

- ID – `driver.findElement(By.id("elementId"))`
- Name – `driver.findElement(By.name("elementName"))`
- Class Name – `driver.findElement(By.className("className"))`
- Tag Name – `driver.findElement(By.tagName("input"))`
- Link Text – `driver.findElement(By.linkText("Click Here"))`
- Partial Link Text – `driver.findElement(By.partialLinkText("Click"))`
- CSS Selector – `driver.findElement(By.cssSelector("input#id"))`
- XPath – `driver.findElement(By.xpath("//input[@id='id']"))`

Q9.

[Freshers]

What is an exception test in Selenium?

An exception test is a test case that verifies that the application throws an expected exception under certain conditions. In Selenium with JUnit/TestNG, you can use annotations to expect specific exceptions:

// JUnit example:

```
@Test(expected = NoSuchElementException.class)
public void testElement() {
    driver.findElement(By.id("nonExistentId"));
}
```

// TestNG example:

```
@Test(expectedExceptions = NoSuchElementException.class)
public void testElement() {
    driver.findElement(By.id("nonExistentId"));
}
```

Q10.

[Freshers]

What is XPath in Selenium? Explain XPath Absolute and XPath Relative.

XPath is a query language used to navigate through elements and attributes in an XML/HTML document. In Selenium, it is used to locate web elements.

XPath Absolute:

- Starts from the root of the HTML document

- Begins with a single slash /
- Example: /html/body/div[1]/form/input[1]
- Fragile – breaks if the page structure changes

XPath Relative:

- Can start from anywhere in the document
- Begins with double slash //
- Example: //input[@id='username']
- More robust and preferred in automation

Q11.

[Freshers]

In XPath, what is the difference between '/' and '//'?

/ (Single slash):

- Selects from the root node
- Creates an absolute path
- Example: /html/body/div — navigates strictly from root

// (Double slash):

- Selects nodes anywhere in the document
- Creates a relative path
- Example: //div[@class='main'] — finds any div with class 'main' regardless of location
- More flexible and commonly used in Selenium

Q12.

[Freshers]

What is the difference between the commands 'type' and 'typeAndWait'?

type:

- Enters text into a text field
- Does not wait for any page load or action after typing
- Used when typing does not trigger a page reload

typeAndWait:

- Enters text into a text field AND waits for the page to reload/load
- Used when typing triggers a page refresh or form submission
- Useful in Selenium IDE for synchronizing actions

Q13.

[Freshers]

Differentiate between findElement() and findElements().

findElement():

- Returns a single WebElement (the first matching element)
- Throws NoSuchElementException if no element is found
- Return type: WebElement

- Example: `driver.findElement(By.id("submit"))`

`findElements()`:

- Returns a list of all matching `WebElements`
 - Returns an empty list if no elements are found (no exception)
 - Return type: `List<WebElement>`
- Example: `driver.findElements(By.className("item"))`

Q14.

[Freshers]

In Selenium, how will you wait until a web page has been loaded completely?

Several approaches:

1. Implicit Wait:

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

2. Explicit Wait with expected condition:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
wait.until(webDriver -> ((JavascriptExecutor) webDriver)
```

```
.executeScript("return document.readyState").equals("complete"));
```

3. PageLoadTimeout:

```
driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
```

4. Using JavascriptExecutor:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
```

```
js.executeScript("return document.readyState").equals("complete");
```

Q15.

[Freshers]

What is Selenium WebDriver?

Selenium WebDriver is a browser automation framework that accepts and sends commands to a browser. It interacts directly with the browser using browser-native support (browser drivers like `ChromeDriver`, `GeckoDriver`, etc.), without needing an intermediate server.

Key features:

- Supports multiple browsers (Chrome, Firefox, Safari, Edge, IE)
- Supports multiple programming languages
- Provides a rich API for interacting with web elements
- Faster and more reliable than Selenium RC
- Part of the Selenium 2.0 and later releases

Q16.

[Freshers]

Is Selenium WebDriver a library?

Yes, Selenium WebDriver is a library (or collection of libraries/bindings) that provides APIs to interact with web browsers programmatically. It is available as language-specific bindings:

- Java: selenium-java.jar
- Python: selenium package (pip install selenium)
- C#: Selenium.WebDriver NuGet package
- Ruby: selenium-webdriver gem

It is not a standalone application but a library you include in your test project.

Q17.

[Freshers]

Is Selenium WebDriver an interface or a class?

WebDriver is an interface in Selenium (in Java). It defines the contract/methods that all browser driver implementations must follow.

Concrete implementations include:

- ChromeDriver – implements WebDriver for Chrome
- FirefoxDriver – implements WebDriver for Firefox
- EdgeDriver – implements WebDriver for Edge
- SafariDriver – implements WebDriver for Safari

This allows polymorphism:

```
WebDriver driver = new ChromeDriver(); // interface reference
```

Q18.

[Freshers]

What are the different types of WebDriver APIs in Selenium?

- ChromeDriver – for Google Chrome
- FirefoxDriver (GeckoDriver) – for Mozilla Firefox
- EdgeDriver – for Microsoft Edge
- SafariDriver – for Apple Safari
- InternetExplorerDriver – for Internet Explorer (deprecated)
- HtmlUnitDriver – headless browser, fast but limited
- PhantomJSDriver – headless browser (deprecated)
- RemoteWebDriver – for running tests on remote machines/Selenium Grid

Q19.

[Freshers]

What programming languages does Selenium WebDriver support?

Selenium WebDriver officially supports:

- Java
- Python

- C# (.NET)
- Ruby
- JavaScript (Node.js)
- Kotlin (via Java bindings)

Community-supported bindings also exist for PHP and other languages.

Q20.

[Freshers]

What open-source frameworks does Selenium WebDriver support?

- TestNG – Java testing framework with parallel execution, data-driven testing
- JUnit – Java unit testing framework
- NUnit – .NET testing framework
- pytest – Python testing framework
- RSpec – Ruby testing framework
- Mocha/Jasmine – JavaScript testing frameworks
- Cucumber/BDD – Behavior-Driven Development
- Maven/Gradle – Build tools for dependency management
- Jenkins/GitHub Actions – CI/CD integration

Q21.

[Freshers]

What is WebDriver's super interface?

The super interface of WebDriver is SearchContext.

Hierarchy:

SearchContext (top-level interface)

■■■ WebDriver (extends SearchContext)

■■■ ChromeDriver

■■■ FirefoxDriver

■■■ ...

SearchContext defines the findElement() and findElements() methods, which WebDriver inherits and extends with browser-control methods like get(), close(), quit(), etc.

Q22.

[Freshers]

Explain: `WebDriver driver = new FirefoxDriver();`

This line creates an instance of FirefoxDriver and assigns it to a reference variable of type WebDriver (interface).

- WebDriver is the interface (left side) – provides polymorphism
- FirefoxDriver is the concrete class (right side) – launches Firefox browser
- driver is the reference variable used to call browser methods

Benefits of using the interface type:

- Easily switch browsers by changing only the right side:

```
WebDriver driver = new ChromeDriver();
```

- Promotes loose coupling
- Follows the Liskov Substitution Principle

Q23.

[Freshers]

Is it necessary to use Selenium Server to run Selenium WebDriver scripts?

No, it is NOT necessary to use Selenium Server (Selenium RC) to run WebDriver scripts locally. WebDriver communicates directly with the browser using browser drivers (ChromeDriver, GeckoDriver, etc.).

Selenium Server IS required when:

- Running tests on a remote machine
- Using Selenium Grid for parallel/distributed testing
- Using RemoteWebDriver to connect to a Grid hub

For local execution, just set up the appropriate browser driver and run your scripts directly.

Q24.

[Freshers]

What will happen if I execute: `driver.get('www.interviewbit.com')`?

It will throw an exception or fail to load the page. The `driver.get()` method requires a fully qualified URL including the protocol (`http://` or `https://`).

Incorrect: `driver.get("www.interviewbit.com")`

Correct: `driver.get("https://www.interviewbit.com")`

Without the protocol, the WebDriver may throw a `WebDriverException` or the browser may not navigate correctly because it cannot determine how to handle the address.

Q25.

[Freshers]

What is an alternative to `driver.get()` for opening a URL?

```
driver.navigate().to("https://www.example.com");
```

Both `driver.get()` and `driver.navigate().to()` load a URL, but `navigate()` provides additional capabilities:

- `driver.navigate().back()` – goes to the previous page
- `driver.navigate().forward()` – goes forward in history
- `driver.navigate().refresh()` – refreshes the current page
- `driver.navigate().to(url)` – navigates to a URL (same as `get()` functionally)

Q26.

[Freshers]

What is the difference between `driver.get()` and `driver.navigate().to()`?

`driver.get(url):`

- Loads a new web page in the current window
- Waits for the page to fully load before proceeding
- Does not maintain browser history for forward/back navigation in the script

`driver.navigate().to(url):`

- Also loads a new web page
- Maintains browser history – you can use `navigate().back()` and `navigate().forward()`
- Slightly more flexible for navigation scenarios

In practice, they behave similarly for loading pages, but `navigate()` is part of a richer navigation API.

Q27.

[Freshers]

What are the differences between `driver.close()` and `driver.quit()`?

`driver.close():`

- Closes only the currently focused browser window/tab
- If it's the only window, the browser process may still run
- Does not end the WebDriver session completely

`driver.quit():`

- Closes ALL browser windows opened by the WebDriver session
- Properly ends the WebDriver session
- Releases all associated resources and driver processes
- Always prefer `quit()` at the end of a test to avoid memory leaks

Q28.

[Freshers]

What are some cases that Selenium WebDriver cannot automate?

- Desktop (non-browser) applications
- Windows-based pop-up dialogs (file upload/download dialogs natively)
- CAPTCHA verification
- Barcode readers
- OTP verification (dynamic one-time passwords)
- Flash/Silverlight-based content
- Video/audio streaming verification
- Biometric authentication
- Mobile native apps (need Appium instead)
- Performance/load testing (need JMeter, etc.)

Q29.

[Freshers]

In Selenium WebDriver, what is an Object Repository?

An Object Repository is a centralized store that holds the locators and properties of web elements used in tests. Instead of hardcoding locators in test scripts, they are stored in one place (e.g., .properties file, Excel, XML, or Page Object classes).

Benefits:

- Easy maintenance – update locators in one place
- Reusability across multiple test cases
- Separation of test logic from UI details
- Cleaner, more readable code

In Java Selenium, the Page Object Model (POM) acts as an object repository.

Q30.

[Experienced]

State the major difference between 'assert' and 'verify' commands.

assert:

- If the assertion fails, the test execution STOPS immediately
- The remaining steps in the test are not executed
- Used when a failure at that point means the test cannot continue meaningfully
- Example: `assert title == "Home Page"` → stops if title doesn't match

verify:

- If verification fails, the failure is logged but test execution CONTINUES
- All remaining steps are executed regardless
- Used for non-critical checks where you want to see all failures at once
- Example: `verify text == "Welcome"` → logs failure but proceeds

Q31.

[Experienced]

Explain the same-origin policy and how Selenium handles it.

Same-Origin Policy (SOP) is a browser security mechanism that prevents a script from one origin (protocol + domain + port) from accessing resources on another origin.

Example: A script at `http://example.com` cannot access `http://other.com`.

How Selenium RC handled it:

- Selenium RC used a proxy server to inject JavaScript into the browser, effectively bypassing SOP by making requests appear same-origin.

How Selenium WebDriver handles it:

- WebDriver uses browser-native automation APIs (not JavaScript injection), so it is largely not affected by SOP.
- WebDriver communicates with the browser driver (ChromeDriver, etc.) outside the browser's JavaScript sandbox.

Q32.

[Experienced]

Explain the pause feature in Selenium IDE.

The Pause feature in Selenium IDE allows you to pause the test execution at a specific point for a given duration (in milliseconds).

Usage:

- Command: pause
- Target: (empty)
- Value: time in milliseconds (e.g., 3000 for 3 seconds)

Purpose:

- Useful for debugging tests by pausing at certain steps
- Can help synchronize with slow-loading elements (though explicit waits are preferred)
- Can observe application state at a specific point during replay

Note: Overusing pause is an anti-pattern; prefer dynamic waits for robust tests.

Q33.

[Experienced]

How do we create right-click and mouse hover actions in Selenium?

Using the Actions class:

Right-click (Context Click):

```
Actions actions = new Actions(driver);
WebElement element = driver.findElement(By.id("elementId"));
actions.contextClick(element).perform();
```

Mouse Hover:

```
Actions actions = new Actions(driver);
WebElement element = driver.findElement(By.id("menuItem"));
actions.moveToElement(element).perform();
```

Double Click:

```
actions.doubleClick(element).perform();
```

Click and Hold:

```
actions.clickAndHold(element).perform();
```

The Actions class is part of org.openqa.selenium.interactions package.

Q34.

[Experienced]

Can we handle a Windows-based pop-up in Selenium?

Selenium WebDriver cannot natively handle Windows-based (OS-level) pop-ups like file upload dialogs, authentication dialogs, or print dialogs.

Workarounds:

1. AutoIT – Windows automation tool that can interact with OS dialogs
2. Robot class (Java) – simulates keyboard/mouse events

```
Robot robot = new Robot();  
robot.keyPress(KeyEvent.VK_ENTER);
```

3. SendKeys for file upload (special case):

```
driver.findElement(By.id("fileInput")).sendKeys("C:/path/to/file.txt");
```

4. Sikuli – image-based automation
5. PyAutoGUI (Python) – for OS-level automation

Q35.

[Experienced]

Can you capture a screenshot using Selenium? Write the code.

Yes, Selenium WebDriver can capture screenshots.

Java:

```
// Cast WebDriver to TakesScreenshot  
TakesScreenshot ts = (TakesScreenshot) driver;  
File source = ts.getScreenshotAs(OutputType.FILE);  
File dest = new File("screenshot.png");  
  
FileUtils.copyFile(source, dest); // Apache Commons IO
```

Python:

```
driver.save_screenshot("screenshot.png")  
  
# Or:  
from selenium.webdriver.common.action_chains import ActionChains  
  
driver.get_screenshot_as_file("screenshot.png")  
  
// For a specific element (Selenium 4):  
WebElement element = driver.findElement(By.id("header"));  
File screenshot = element.getScreenshotAs(OutputType.FILE);
```

Q36.

[Experienced]

Explain different types of frameworks and connection of Selenium with Robot Framework.

Types of Automation Frameworks:

1. Data-Driven Framework – test data stored externally (Excel, CSV); same script runs with different data
2. Keyword-Driven Framework – test actions represented as keywords; non-technical users can write tests
3. Hybrid Framework – combination of data-driven and keyword-driven

4. Page Object Model (POM) – UI pages represented as classes; separates test logic from page details
5. BDD Framework – tests written in Gherkin (Given-When-Then); Cucumber integration

Robot Framework with Selenium:

- Robot Framework is a keyword-driven test automation framework
- Uses SeleniumLibrary plugin to integrate with Selenium WebDriver
- Install: `pip install robotframework-seleniumlibrary`
- Tests written in plain English keywords:

Open Browser `https://example.com` `chrome`

Input Text `id=username` `admin`

Click Button `id=login`

Q37.

[Experienced]

Demonstrate usage of Selenium through a test application.

Example: Login test using Java + Selenium WebDriver + TestNG

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;

public class LoginTest {
    WebDriver driver;

    @BeforeMethod
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://example.com/login");
    }

    @Test
    public void testLogin() {
        driver.findElement(By.id("username")).sendKeys("admin");
        driver.findElement(By.id("password")).sendKeys("password123");
        driver.findElement(By.id("loginBtn")).click();
        String title = driver.getTitle();
        assert title.equals("Dashboard") : "Login failed!";
    }

    @AfterMethod
    public void tearDown() {
        driver.quit();
    }
}
```

Q38.

[Experienced]

Explain the 7 basic steps of Selenium testing and widely used commands.

7 Basic Steps of Selenium Testing:

1. Create a WebDriver instance

```
WebDriver driver = new ChromeDriver();
```

2. Navigate to a URL

```
driver.get("https://example.com");
```

3. Locate web elements

```
WebElement element = driver.findElement(By.id("username"));
```

4. Perform actions on elements

```
element.sendKeys("admin");  
driver.findElement(By.id("loginBtn")).click();
```

5. Capture and verify results

```
String title = driver.getTitle();  
assert title.equals("Dashboard");
```

6. Handle synchronization (waits)

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("welcome")));
```

7. Close the browser

```
driver.quit();
```

Q39.

[Experienced]

What do you understand about Page Object Model (POM)? What are its advantages?

Page Object Model (POM) is a design pattern where each web page is represented as a class. The class contains web element locators and methods that perform actions on those elements.

Structure:

```
public class LoginPage {  
    WebDriver driver;
```

```
    @FindBy(id = "username")  
    WebElement usernameField;
```

```
    @FindBy(id = "loginBtn")  
    WebElement loginButton;
```

```
    public void login(String user, String pass) {  
        usernameField.sendKeys(user);  
        loginButton.click();  
    }
```

```
}  
}
```

Advantages:

- Reduces code duplication
- Easy maintenance – update locators in one place
- Improves readability and reusability
- Separates test logic from page interactions
- Scales well for large test suites

Q40.

[Experienced]

What is Jenkins and what are the benefits of using it with Selenium?

Jenkins is an open-source CI/CD automation server that automates building, testing, and deploying applications.

Benefits with Selenium:

- Automated test execution – trigger Selenium tests on every code commit
- Scheduled runs – run regression suites at scheduled intervals
- Parallel execution – run multiple test jobs simultaneously
- Reporting – integrate with TestNG/Allure reports in Jenkins dashboard
- Notifications – email/Slack alerts on test failures
- Environment management – run tests across different environments
- Integration – works with Git, Maven, Docker, and more
- History – track test results and trends over time

Q41.

[Experienced]

How do you select a date from a datepicker in a webpage using Selenium?

Approach 1: Direct input using sendKeys (if text input is allowed)

```
WebElement dateField = driver.findElement(By.id("datepicker"));  
dateField.sendKeys("04/20/2026");
```

Approach 2: JavaScript injection

```
JavascriptExecutor js = (JavascriptExecutor) driver;  
js.executeScript("document.getElementById('datepicker').value='2026-04-20'");
```

Approach 3: Click through the calendar UI

```
driver.findElement(By.id("datepicker")).click();  
// Navigate month/year as needed  
driver.findElement(By.xpath("//td[@data-date='20']").click();
```

Approach 4: Clear and type

```
WebElement date = driver.findElement(By.id("datepicker"));
date.clear();
date.sendKeys("04/20/2026");
```

Q42.

[Experienced]

What are broken links? How can you detect them in Selenium?

Broken links are hyperlinks that point to pages/resources that no longer exist (returns HTTP 4xx or 5xx errors).

Detection using Selenium + HttpURLConnection (Java):

```
List<WebElement> links = driver.findElements(By.tagName("a"));
for (WebElement link : links) {
    String url = link.getAttribute("href");
    if (url != null && !url.isEmpty()) {
        HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
        conn.setRequestMethod("HEAD");
        conn.connect();
        int responseCode = conn.getResponseCode();
        if (responseCode >= 400) {
            System.out.println("Broken: " + url + " - " + responseCode);
        }
    }
}
```

HTTP 404 = Not Found, 500 = Server Error are common broken link indicators.

Q43.

[Experienced]

What is a window handle? How do you handle multiple windows in Selenium?

A window handle is a unique identifier (string) assigned to each browser window/tab opened by WebDriver.

Handling multiple windows:

```
// Get the main window handle
String mainWindow = driver.getWindowHandle();

// Get all window handles
Set<String> allWindows = driver.getWindowHandles();

// Switch to a new window
for (String window : allWindows) {
    if (!window.equals(mainWindow)) {
        driver.switchTo().window(window);
        break;
    }
}
```

```
// Perform actions in new window...
System.out.println(driver.getTitle());
```

```
// Switch back to main window
driver.switchTo().window(mainWindow);
```

Q44.

[WebDriver]

How to create an Object Repository in your project?

Methods to create an Object Repository:

1. Using .properties file:

```
# locators.properties
username=id=username
password=id=password
loginBtn=xpath=//button[@id='login']
```

```
// Read in Java using Properties class
```

2. Using Page Object Model (POM):

```
public class LoginPage {
    @FindBy(id = "username")
    WebElement usernameField;
}
```

3. Using Excel/CSV:

- Store element name, locator type, locator value
- Read using Apache POI or OpenCSV

4. Using XML:

```
<object name="username" type="id" value="username"/>
```

- Parse using DOM/SAX parser

POM is the most popular and recommended approach.

Q45.

[WebDriver]

What are the different types of waits that WebDriver supports?

1. Implicit Wait:

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

- Applies globally to all findElement calls
- Waits for the specified duration before throwing NoSuchElementException

2. Explicit Wait:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("btn")));
```

- Waits for a specific condition before proceeding
- More flexible and targeted

3. Fluent Wait:

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofSeconds(5))
```

```
.ignoring(NoSuchElementException.class);
```

- Customizable polling interval and exceptions to ignore

4. Thread.sleep() (not recommended):

```
Thread.sleep(3000); // Hard wait – avoid in production tests
```

Q46.

[WebDriver]

Mention the different types of navigation commands.

```
driver.navigate().to(url) - Navigate to a specific URL
driver.navigate().back() - Go back to the previous page (browser back button)
driver.navigate().forward() - Go forward in browser history (browser forward button)
driver.navigate().refresh() - Refresh/reload the current page
```

Difference from driver.get():

- driver.get() only loads a URL
- driver.navigate() provides all of the above navigation capabilities
- navigate().to() and get() behave similarly but navigate() preserves history for back/forward

Q47.

[WebDriver]

How does Selenium WebDriver interact with the browser?

Selenium WebDriver Architecture:

1. Test Script sends command to WebDriver API
2. WebDriver API communicates with the Browser Driver (e.g., ChromeDriver, GeckoDriver)
3. Browser Driver is a separate executable that implements the W3C WebDriver protocol
4. Browser Driver translates commands into browser-native actions
5. Browser executes the action and returns the response
6. Response travels back through the same chain to the test script

Key Points:

- Communication uses HTTP/JSON Wire Protocol (W3C WebDriver standard)
- Each browser has its own driver (ChromeDriver for Chrome, GeckoDriver for Firefox, etc.)
- No intermediate server needed for local execution

Q48.

[WebDriver]

What are Selenium WebDriver Listeners?

WebDriver Listeners are interfaces that allow you to intercept and listen to WebDriver events. They enable you to add custom behavior before/after WebDriver actions.

Types:

1. `WebDriverEventListener` (deprecated in Selenium 4)
 - Interface with methods like `beforeClickOn`, `afterClickOn`, `onException`, etc.
2. `EventFiringWebDriver` (deprecated in Selenium 4)
 - Wraps a `WebDriver` instance to fire events
3. `WebDriverListener` (Selenium 4+)
 - New interface using `@Override` methods
 - More flexible and annotation-based

Use cases:

- Logging every WebDriver action
- Taking screenshots on failure
- Highlighting elements before interaction
- Custom reporting

Q49.

[WebDriver]

What is the implementation of WebDriver Listeners?

Selenium 4 approach using `WebDriverListener`:

```
// Step 1: Create listener class
public class MyListener implements WebDriverListener {
    @Override
    public void beforeGet(WebDriver driver, String url) {
        System.out.println("Navigating to: " + url);
    }

    @Override
    public void afterClick(WebElement element) {
        System.out.println("Clicked on: " + element);
    }

    @Override
    public void onException(Throwable throwable, Object cmd) {
        System.out.println("Exception: " + throwable.getMessage());
    }
}

// Step 2: Decorate the driver
WebDriver driver = new ChromeDriver();
WebDriver decoratedDriver = new EventFiringDecorator<>(new MyListener()).decorate(driver);

// Use decoratedDriver in tests
decoratedDriver.get("https://example.com");
```

Q50.

[WebDriver]

How do you handle AJAX calls in Selenium WebDriver?

AJAX (Asynchronous JavaScript And XML) calls load content dynamically without page refresh, making synchronization challenging.

Approaches:

1. Explicit Wait with ExpectedConditions:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("result")));
```

2. Fluent Wait (polls repeatedly):

```
Wait<WebDriver> wait = new FluentWait<>(driver)  
.withTimeout(Duration.ofSeconds(20))  
.pollingEvery(Duration.ofSeconds(2))  
  
.ignoring(NoSuchElementException.class);  
WebElement result = wait.until(d -> d.findElement(By.id("ajaxResult")));
```

3. JavaScript Executor to check jQuery:

```
((JavascriptExecutor) driver).executeScript("return jQuery.active == 0");
```

4. Check element presence/visibility after AJAX loads

Q51.

[WebDriver]

Which WebDriver implementation is the fastest?

HtmlUnitDriver is the fastest WebDriver implementation because it is a headless browser – it doesn't render a visible UI.

Comparison:

- HtmlUnitDriver – fastest (headless, Java-based, no actual browser)
- PhantomJS Driver – fast (headless, deprecated)
- ChromeDriver (headless mode) – fast for headless
- ChromeDriver (regular) – standard speed
- FirefoxDriver – slightly slower than Chrome in most benchmarks

For CI/CD environments, Chrome in headless mode is the most popular balance of speed and compatibility:

```
ChromeOptions options = new ChromeOptions();  
options.addArguments("--headless");  
WebDriver driver = new ChromeDriver(options);
```

Q52.

[WebDriver]

At minimum, how many parameters do Selenium commands have?

Selenium commands have a minimum of 0 parameters, but the structure varies:

In Selenium IDE (Selenese), commands have:

- Command (required) – the action to perform
- Target (optional) – the locator or element
- Value (optional) – additional input

Examples:

- open / https://example.com / (0 extra params beyond target)
- click / id=button / (no value needed)
- type / id=input / Hello (command + target + value = 3 parts)

In WebDriver API (Java), method parameters vary:

- driver.getTitle() – 0 parameters
- driver.getUrl() – 1 parameter
- driver.findElement(By.id("x")) – 1 parameter

Q53.

[WebDriver]

Why declare `WebDriver driver = new FirefoxDriver()` instead of `FirefoxDriver driver = new FirefoxDriver()`?

Using the interface type (WebDriver) instead of the concrete class (FirefoxDriver) promotes:

1. Polymorphism / Loose Coupling:

```
WebDriver driver = new ChromeDriver(); // Easy to switch browsers
// Just change one line to switch from Firefox to Chrome
```

2. Flexibility:

- Can be changed to any browser driver without modifying test logic
- Enables browser-switching through configuration files

3. Better OOP Design:

- Follows "Program to an interface, not an implementation"
- Reduces dependency on specific browser implementations

4. Supports Factory Pattern:

```
public WebDriver getDriver(String browser) {
    if (browser.equals("chrome")) return new ChromeDriver();
    if (browser.equals("firefox")) return new FirefoxDriver();
}
```

Q54.

[WebDriver]

What kinds of Selenium WebDriver exceptions have you encountered?

Common WebDriver Exceptions:

- NoSuchElementException – Element not found with given locator
- StaleElementReferenceException – Element is no longer attached to DOM
- TimeoutException – Operation exceeded the wait time
- ElementNotVisibleException – Element present in DOM but not visible
- ElementNotInteractableException – Element exists but cannot be interacted with
- WebDriverException – General WebDriver error
- InvalidSelectorException – Locator syntax is invalid
- NoSuchWindowException – Target window doesn't exist
- NoSuchFrameException – Frame/iframe not found
- NoAlertPresentException – Expected alert not present
- MoveTargetOutOfBoundsException – Target is outside the viewport
- SessionNotCreatedException – Browser session could not be created

Q55.

[WebDriver]

What is the best way to deal with StaleElementReferenceException?

StaleElementReferenceException occurs when the referenced element is no longer attached to the DOM (due to page refresh, dynamic content reload, etc.).

Solutions:

1. Re-locate the element before use:

```
WebElement element = driver.findElement(By.id("btn"));
element.click(); // If stale, re-find
driver.findElement(By.id("btn")).click();
```

2. Use try-catch with retry logic:

```
for (int i = 0; i < 3; i++) {
    try {
        driver.findElement(By.id("btn")).click();
        break;
    } catch (StaleElementReferenceException e) {
        // Retry
    }
}
```

3. Use Explicit Wait:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.refreshed(
    ExpectedConditions.elementToBeClickable(By.id("btn"))));
```

4. Use Page Object Model with fresh lookups

Q56.

[WebDriver]

How do you take a screenshot in Selenium WebDriver?

Java:

```
TakesScreenshot ts = (TakesScreenshot) driver;  
File src = ts.getScreenshotAs(OutputType.FILE);  
FileUtils.copyFile(src, new File("screenshot.png"));
```

Python:

```
driver.save_screenshot("screenshot.png")
```

Selenium 4 – Screenshot of specific element:

```
WebElement element = driver.findElement(By.id("header"));  
File elementScreenshot = element.getScreenshotAs(OutputType.FILE);  
FileUtils.copyFile(elementScreenshot, new File("element.png"));
```

Using Base64 (for embedding in reports):

```
String base64 = ts.getScreenshotAs(OutputType.BASE64);
```

On test failure (TestNG listener):

```
@Override  
public void onTestFailure(ITestResult result) {  
    // Take screenshot and attach to report  
}
```

Q57.

[WebDriver]

How do I use Selenium WebDriver to enter text into a text box?

Using sendKeys():

```
WebElement textBox = driver.findElement(By.id("username"));  
textBox.sendKeys("myUsername");
```

Clear existing text first:

```
textBox.clear();  
textBox.sendKeys("newText");
```

Using Actions class:

```
Actions actions = new Actions(driver);  
actions.click(textBox).sendKeys("text").perform();
```

Using JavascriptExecutor:

```
JavascriptExecutor js = (JavascriptExecutor) driver;  
js.executeScript("arguments[0].value='myText'", textBox);
```

Python:

```
text_box = driver.find_element(By.ID, "username")
text_box.send_keys("myUsername")
```

Q58.

[WebDriver]

How can I clear the text in a text box using Selenium WebDriver?

Method 1 – clear():

```
WebElement textBox = driver.findElement(By.id("username"));
textBox.clear();
```

Method 2 – Select all and delete:

```
textBox.sendKeys(Keys.CONTROL + "a");
textBox.sendKeys(Keys.DELETE);
```

Method 3 – Using Actions:

```
Actions actions = new Actions(driver);
actions.click(textBox)
    .keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL)
    .sendKeys(Keys.DELETE)
    .perform();
```

Method 4 – JavaScript:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].value=''", textBox);
```

Note: clear() triggers the change/input event on most browsers.

Q59.

[WebDriver]

What is the best way to get the text of a web element?

Using getText():

```
WebElement element = driver.findElement(By.id("message"));
String text = element.getText();
System.out.println(text);
```

For input field values (getText() returns empty for inputs):

```
String value = element.getAttribute("value");
```

For hidden elements:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
String text = (String) js.executeScript("return arguments[0].textContent", element);
```

Python:

```
element = driver.find_element(By.ID, "message")
text = element.text
print(text)
```

Note: `getText()` returns visible text only. Use `getAttribute("textContent")` or `"innerHTML"` for hidden text.

Q60.

[WebDriver]

How do I retrieve the value of an attribute in Selenium WebDriver?

Using `getAttribute()`:

```
WebElement element = driver.findElement(By.id("myInput"));
```

```
String value = element.getAttribute("value"); // Input value
String href = element.getAttribute("href"); // Link URL
String className = element.getAttribute("class"); // CSS class
String isChecked = element.getAttribute("checked"); // Checkbox state
```

```
String placeholder = element.getAttribute("placeholder");
```

```
// Selenium 4+ - getDomAttribute() for actual DOM attribute:
String attr = element.getDomAttribute("value");
```

```
// Selenium 4+ - getDomProperty() for DOM property:
String prop = element.getDomProperty("value");
```

Python:

```
element = driver.find_element(By.ID, "myInput")
value = element.get_attribute("value")
```

Q61.

[WebDriver]

How can I click on a hyperlink using Selenium WebDriver?

Method 1 – By Link Text:

```
driver.findElement(By.linkText("Click Here")).click();
```

Method 2 – By Partial Link Text:

```
driver.findElement(By.partialLinkText("Click")).click();
```

Method 3 – By href attribute:

```
driver.findElement(By.xpath("//a[@href='/page']")).click();
```

Method 4 – By CSS Selector:

```
driver.findElement(By.cssSelector("a.nav-link")).click();
```

Method 5 – Using Actions:

```
WebElement link = driver.findElement(By.linkText("Home"));
Actions actions = new Actions(driver);
actions.click(link).perform();
```

Method 6 – JavaScript click (for hidden/overlapping elements):

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].click()", link);
```

Q62.

[WebDriver]

How do I submit a form using Selenium WebDriver?

Method 1 – submit() method:

```
driver.findElement(By.id("formElement")).submit();
```

Method 2 – Click the submit button:

```
driver.findElement(By.id("submitBtn")).click();
```

Method 3 – Press ENTER key:

```
driver.findElement(By.id("inputField")).sendKeys(Keys.ENTER);
```

Method 4 – JavaScript:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("document.getElementById('myForm').submit()");
```

Note: submit() can be called on any element within the form, not just the submit button. However, using click() on the actual button is more realistic and recommended.

Q63.

[WebDriver]

How do I press the ENTER key on a text box in Selenium WebDriver?

Method 1 – Using Keys.ENTER with sendKeys:

```
WebElement textBox = driver.findElement(By.id("searchBox"));
textBox.sendKeys(Keys.ENTER);
```

Method 2 – Press ENTER after typing:

```
textBox.sendKeys("search query" + Keys.ENTER);
// or
textBox.sendKeys("search query");
textBox.sendKeys(Keys.ENTER);
```

Method 3 – Using Actions class:

```
Actions actions = new Actions(driver);
actions.click(textBox).sendKeys(Keys.ENTER).perform();
```

Method 4 – Using Keys.RETURN:

```
textBox.sendKeys(Keys.RETURN);
```

Note: Keys.ENTER and Keys.RETURN usually produce the same result for form submissions.

Q64.

[WebDriver]

How can I mouse hover over a web element using WebDriver?

Using Actions class – moveToElement():

```
WebElement element = driver.findElement(By.id("menu"));
Actions actions = new Actions(driver);
actions.moveToElement(element).perform();
```

Hover then click:

```
actions.moveToElement(element).click().perform();
```

Hover with offset (move to specific coordinates):

```
actions.moveToElement(element, 10, 5).perform();
```

Using JavascriptExecutor (for elements with CSS :hover):

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript(
    "var event = new MouseEvent('mouseover', {bubbles: true});" +
    "arguments[0].dispatchEvent(event);", element);
```

Python:

```
from selenium.webdriver.common.action_chains import ActionChains
actions = ActionChains(driver)
actions.move_to_element(element).perform()
```

Q65.

[WebDriver]

How do you use the Recovery Scenario in Selenium WebDriver?

Recovery Scenarios handle unexpected situations (pop-ups, exceptions, alerts) during test execution.

Common Recovery Approaches:

1. Try-Catch blocks:

```
try {
    driver.findElement(By.id("element")).click();
}
```

```
} catch (NoSuchElementException e) {  
// Recovery: wait and retry  
Thread.sleep(1000);  
driver.findElement(By.id("element")).click();  
}  
}
```

2. Handle unexpected alerts:

```
try {  
driver.switchTo().alert().accept();  
} catch (NoAlertPresentException e) {  
// No alert, continue normally  
}  
}
```

3. TestNG @AfterMethod for cleanup:

```
@AfterMethod  
public void recoverAfterTest(ITestResult result) {  
if (result.getStatus() == ITestResult.FAILURE) {  
driver.navigate().to(baseUrl); // Reset to known state  
}  
}
```

4. Use Page Object with re-usable recovery methods

Q66.

[WebDriver]

What is the Selenium WebDriver Architecture?

Selenium WebDriver Architecture has 4 main components:

1. Selenium Client Libraries (Language Bindings):
 - Test scripts written in Java, Python, C#, etc.
 - Provide the API to write tests
2. W3C WebDriver Protocol / JSON Wire Protocol:
 - Communication protocol between client and browser driver
 - HTTP requests with JSON payloads
 - Standardized as W3C WebDriver spec
3. Browser Drivers:
 - Executable specific to each browser
 - ChromeDriver (Chrome), GeckoDriver (Firefox), EdgeDriver (Edge)
 - Translates WebDriver commands into browser-specific actions
4. Browsers:
 - The actual web browser (Chrome, Firefox, Edge, Safari)
 - Executes the commands from the browser driver

Flow: Test Script → WebDriver API → HTTP/JSON → Browser Driver → Browser

Q67.

[Tricky]

Why is it important to use TestNG when working with Selenium RC?

TestNG provides essential features that complement Selenium RC:

- Test Grouping – organize tests into groups (smoke, regression)
- Parallel Execution – run multiple tests simultaneously
- Data Provider – data-driven testing with `@DataProvider`
- Annotations – `@Test`, `@BeforeMethod`, `@AfterMethod`, `@BeforeClass` etc.
- HTML Reports – built-in detailed reporting
- Dependency Testing – specify test dependencies with `dependsOnMethods`
- Assertions – rich assertion library
- Retry Failed Tests – automatic retry mechanism
- Priority – control test execution order with priority attribute
- Suite XML – configure entire test suites via `testng.xml`

These features significantly enhance test organization, maintenance, and reporting when combined with Selenium.

Q68.

[Tricky]

Can Selenium be used to test responsive web design?

Yes, Selenium can be used to test responsive web design:

1. Resize browser window:

```
driver.manage().window().setSize(new Dimension(375, 812)); // iPhone size
```

2. Simulate mobile devices using ChromeDriver DevTools:

```
Map<String, Object> deviceMetrics = new HashMap<>();
deviceMetrics.put("width", 375);
deviceMetrics.put("height", 812);
deviceMetrics.put("pixelRatio", 3.0);
Map<String, Object> mobileEmulation = new HashMap<>();
mobileEmulation.put("deviceMetrics", deviceMetrics);
ChromeOptions options = new ChromeOptions();
options.setExperimentalOption("mobileEmulation", mobileEmulation);
```

3. Use Chrome DevTools Protocol (CDP) in Selenium 4

4. Third-party tools: BrowserStack, Sauce Labs for real device testing

Note: True responsive testing requires visual validation tools like Applifools or Percy.

Q69.

[Tricky]

What API should be used to test databases when using Selenium WebDriver?

Selenium WebDriver itself does not provide database testing capabilities. Use standard database APIs:

Java:

- JDBC (Java Database Connectivity) – universal Java DB API

```
Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
```

Python:

- sqlite3 – built-in for SQLite
- psycopg2 – PostgreSQL
- mysql-connector-python – MySQL
- pyodbc – ODBC databases

Other tools:

- Hibernate / JPA – ORM frameworks
- Spring JDBC Template
- Apache DbUtils

Common DB test scenarios with Selenium:

- Verify data after form submission
- Validate test data before/after test
- Clean up test data post-test

Q70.

[Tricky]

What are assertions in Selenium?

Assertions in Selenium verify that the application state matches the expected state. They are provided by testing frameworks like TestNG and JUnit.

Types of Assertions (TestNG):

Hard Assertions (Assert):

- Stops test execution immediately on failure

```
Assert.assertEquals(actual, expected);
Assert.assertTrue(condition);
Assert.assertFalse(condition);
Assert.assertNotNull(object);
```

Soft Assertions (SoftAssert):

- Collects all failures and reports at the end

```
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals(actual, expected);
softAssert.assertTrue(condition);
softAssert.assertAll(); // Must call to report failures
```

Common assertions:

- assertEquals – compare two values
- assertTrue/assertFalse – check boolean condition
- assertNull/assertNotNull – check null state
- assertEquals/assertNotSame – check object identity

Q71.

[Tricky]

What is the Silk Test Tool?

Silk Test is a commercial automated testing tool by Micro Focus (now part of OpenText) used for functional and regression testing of web, mobile, and desktop applications.

Key Features:

- Supports multiple testing environments (web, desktop, mobile)
- Uses Visual Basic-like scripting language (4Test) or Java/C#
- Keyword-driven testing support
- Cross-browser testing
- Mobile testing (iOS, Android)
- Integration with CI/CD tools

Comparison with Selenium:

- Silk Test is commercial (paid); Selenium is free/open-source
- Silk Test has built-in reporting; Selenium needs third-party tools
- Silk Test supports desktop apps; Selenium is web-only
- Selenium has larger community and more language support

Q72.

[Tricky]

What is the purpose of the testng.xml file?

The testng.xml file is the configuration file for TestNG that defines and controls the test suite execution.

Key purposes:

- Define test suites and test groups
- Control which test classes/methods to include or exclude
- Configure parallel execution (methods, classes, tests)
- Set test parameters for data-driven testing
- Define listeners and reporters

Example testng.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "testng.dtd">
```

```
<suite name="Selenium Suite" parallel="classes" thread-count="3">
<test name="Login Tests">
<groups>
<run><include name="smoke"/></run>
</groups>
<classes>
<class name="tests.LoginTest"/>
<class name="tests.DashboardTest"/>
</classes>
</test>
</suite>
```

Q73.

[Tricky]

What are the areas where Selenium can improve its features?

Areas for improvement in Selenium:

- Desktop application testing – currently only supports browsers
- Built-in reporting – no native reports; needs TestNG/Allure/Extent
- Image comparison – no built-in visual testing
- Video recording – no native test recording
- CAPTCHA handling – still not supported
- Better mobile support – Appium is separate
- Faster execution – compared to some commercial tools
- AI-based element detection – self-healing locators
- Better error messages – more descriptive exception details
- Native Windows dialog handling – still requires workarounds
- Improved API testing integration
- Better shadow DOM support (improved in Selenium 4)
- Simplified grid setup (improved with Docker support in Selenium 4)

Q74.

[Tricky]

Can you explain what Page Factory is?

Page Factory is an extension of the Page Object Model (POM) design pattern, provided by Selenium's support library. It uses annotations to initialize web elements automatically.

Key features:

- @FindBy annotation to locate elements
- PageFactory.initElements() to initialize elements
- Lazy initialization – elements are found when first accessed
- Cleaner and more readable code than manual findElement calls

Example:

```
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
```

```
public class LoginPage {
    @FindBy(id = "username")
    WebElement usernameField;
```

```
    @FindBy(id = "loginBtn")
    WebElement loginButton;
```

```
    public LoginPage(WebDriver driver) {
        PageFactory.initElements(driver, this);
    }
```

```
    public void login(String user) {
        usernameField.sendKeys(user);
        loginButton.click();
    }
}
```

Note: Page Factory elements are re-looked up on each access, which can cause StaleElementReferenceException less frequently.

Q75.

[Tricky]

What is the Actions class?

The Actions class in Selenium WebDriver is used to perform complex user interactions and advanced keyboard/mouse operations that cannot be done with simple click() or sendKeys().

Import:

```
import org.openqa.selenium.interactions.Actions;
```

Common Actions:

```
Actions actions = new Actions(driver);
```

```
// Mouse actions
actions.click(element).perform();
actions.doubleClick(element).perform();

actions.rightClick(element).perform(); // contextClick
actions.moveToElement(element).perform(); // hover
```

```
actions.clickAndHold(element).perform();
actions.dragAndDrop(source, target).perform();
```

```
// Keyboard actions
actions.sendKeys(Keys.ENTER).perform();
actions.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();
```

```
// Chaining actions
```

```
actions.moveToElement(menu)
    .click(menuItem)
```

```
.sendKeys("text")  
.perform();
```

build() compiles the actions; perform() executes them.

Q76.

[Tricky]

What are the steps for troubleshooting tests using Selenium IDE?

Troubleshooting steps for Selenium IDE:

1. Check the base URL – ensure it's set correctly in IDE settings
2. Use the Pause/Step feature – step through commands one by one to identify the failing step
3. Check locators – use the IDE's target selector to verify element locators are correct
4. Add explicit waits – insert `waitForElementVisible` or pause commands for dynamic content
5. Check for pop-ups/alerts – handle unexpected alerts before they block execution
6. Review the log panel – Selenium IDE shows detailed command logs and errors
7. Use the debugger – set breakpoints and inspect application state
8. Re-record the failing section – sometimes re-recording gives better locators
9. Update Selenium IDE – ensure you're using the latest version
10. Check browser compatibility – test in different browsers

Q77.

[Tricky]

What steps resolve an issue where a script works on Chrome but not Internet Explorer?

Browser compatibility issues between Chrome and IE:

1. IE-specific settings:
 - Enable Protected Mode for all security zones (Tools → Internet Options → Security)
 - Set zoom level to 100%
 - Disable Enhanced Protected Mode (IE 11)
 - Use `InternetExplorerOptions` to configure:

```
InternetExplorerOptions options = new InternetExplorerOptions();  
options.ignoreZoomSettings();  
options.introduceFlakinessByIgnoringSecurityDomains();
```

2. Check locators – IE may interpret XPath/CSS differently
3. Handle timing issues – IE is generally slower; increase waits
4. JavaScript execution differences – some JS may work in Chrome but not IE
5. Use compatibility-specific waits
6. Check IE driver version matches IE browser version
7. Clear IE cache before running tests
8. Test with 32-bit IE driver even on 64-bit systems

Q78.

[Tricky]

Is it possible to open pop-up windows with Selenium?

Yes, Selenium can handle browser-based pop-up windows:

1. JavaScript alert/confirm/prompt pop-ups:

```
Alert alert = driver.switchTo().alert();  
alert.accept(); // Click OK  
alert.dismiss(); // Click Cancel  
  
alert.sendKeys("text"); // Enter text in prompt  
String alertText = alert.getText();
```

2. New browser window pop-ups:

```
// Handle using window handles  
Set<String> windows = driver.getWindowHandles();  
for (String window : windows) {  
    driver.switchTo().window(window);  
}
```

3. Opening pop-ups:

```
driver.findElement(By.id("openPopupBtn")).click();  
// Then switch to the new window
```

Note: Native OS pop-ups (file dialogs, OS alerts) cannot be handled directly by Selenium – use AutoIT or Robot class instead.

Q79.

[Tricky]

Can Selenium be used to launch web browsers?

Yes, Selenium WebDriver can launch web browsers:

Chrome:

```
WebDriver driver = new ChromeDriver();
```

Firefox:

```
WebDriver driver = new FirefoxDriver();
```

Edge:

```
WebDriver driver = new EdgeDriver();
```

Safari:

```
WebDriver driver = new SafariDriver();
```

Headless Chrome (no visible UI):

```
ChromeOptions options = new ChromeOptions();  
options.addArguments("--headless");  
WebDriver driver = new ChromeDriver(options);
```

Remote browser (Selenium Grid):

```
WebDriver driver = new RemoteWebDriver(  
    new URL("http://localhost:4444/wd/hub"),  
    new ChromeOptions()  
);
```

Selenium 4 + Selenium Manager automatically downloads the required browser driver, so manual driver management is no longer needed in many cases.

Q80.

[Tricky]

Is it possible to use only `perform()` without `build()` in Actions?

Yes, in Selenium WebDriver (especially modern versions), you can use `perform()` directly without calling `build()` first.

Without `build()`:

```
Actions actions = new Actions(driver);  
actions.moveToElement(element).click().perform(); // Valid
```

With `build()` (traditional approach):

```
Actions actions = new Actions(driver);  
Action action = actions.moveToElement(element).click().build();  
action.perform();
```

Explanation:

- build() compiles the action sequence into a single composite Action object
- perform() both builds and performs the action sequence
- In modern Selenium, perform() internally calls build(), so calling build() separately is redundant
- The direct perform() approach is more concise and equally effective

Q81.

[Tricky]

What is StaleElementReferenceException? When does it occur? How to overcome it?

StaleElementReferenceException is thrown when a previously found WebElement is no longer attached to the DOM (document object model).

When it occurs:

- Page is refreshed after finding the element
- DOM is updated (AJAX reload, Angular/React re-render)
- Navigating away and back
- JavaScript modifies or recreates the element

How to overcome:

1. Re-find the element:

```
driver.findElement(By.id("btn")).click(); // Find fresh each time
```

2. Retry mechanism:

```
int attempts = 0;
while (attempts < 3) {
try { driver.findElement(By.id("btn")).click(); break; }

catch (StaleElementReferenceException e) { attempts++; }
}
```

3. Explicit Wait with refreshed condition:

```
wait.until(ExpectedConditions.refreshed(
ExpectedConditions.elementToBeClickable(By.id("btn"))));
```

4. Page Object Model with fresh lookups instead of stored references

Q82.

[Tricky]

In a Selenium script, what happens if you use both implicit and explicit wait?

Using both implicit and explicit waits can lead to unpredictable behavior and longer wait times.

What happens:

- Implicit wait is global and applies to all `findElement()` calls
- Explicit wait is specific to a condition
- When both are active, they can interact:
 - If implicit wait = 10s and explicit wait = 5s
 - The actual wait may exceed 5s due to implicit wait interference
 - Selenium may wait up to 15s in some scenarios

Recommendation:

- Never mix implicit and explicit waits in the same test
- Use explicit waits (`WebDriverWait` + `ExpectedConditions`) for better control
- If implicit wait is set in a base class/setup, avoid explicit waits in tests
- Fluent Wait offers the best control without conflicts

Official Selenium docs warn against combining these waits.

Q83.

[Tricky]

What happens if you use both `Thread.Sleep` and `WebDriver Waits`?

`Thread.sleep()` is a hard/static wait that pauses execution for a fixed duration regardless of whether the element is ready.

Issues with combining `Thread.sleep` and `WebDriver` waits:

- Wastes time – if element loads in 2s but sleep is 5s, 3s is wasted
- Unreliable – if page loads slowly, 5s sleep may not be enough
- Cumulative delays – multiple sleeps add up significantly
- Makes tests fragile and slow

Best practices:

- Avoid `Thread.sleep()` in production test code
- Replace with explicit waits:

```
// Instead of: Thread.sleep(3000);  
// Use:  
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.until(ExpectedConditions.elementToBeClickable(By.id("btn")));
```

- `Thread.sleep()` acceptable only for debugging purposes temporarily

Q84.

[Tricky]

How can I type text into a text box without using `sendKeys()`?

Alternative methods to type text without `sendKeys()`:

1. JavaScript Executor (most common alternative):

```
JavascriptExecutor js = (JavascriptExecutor) driver;
WebElement element = driver.findElement(By.id("inputBox"));
js.executeScript("arguments[0].value='Hello World'", element);
```

2. Actions class:

```
Actions actions = new Actions(driver);
actions.click(element).sendKeys("Hello World").perform();
// Note: Actions.sendKeys internally uses similar mechanism
```

3. Robot class (Java AWT):

```
element.click();
Robot robot = new Robot();
// Type using robot.keyPress() for each character
```

4. Clipboard paste via Robot:

```
StringSelection str = new StringSelection("Hello World");
Toolkit.getDefaultToolkit().getSystemClipboard().setContents(str, null);
robot.keyPress(KeyEvent.VK_CONTROL);
robot.keyPress(KeyEvent.VK_V);
robot.keyRelease(KeyEvent.VK_V);
robot.keyRelease(KeyEvent.VK_CONTROL);
```

Q85.

[Tricky]

How does Selenium WebDriver handle hidden elements?

Selenium WebDriver cannot directly interact with hidden elements (display:none or visibility:hidden). Attempting to click or type on them throws `ElementNotInteractableException`.

Approaches:

1. Make element visible using JavaScript:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].style.display='block'", element);
element.click();
```

2. Set value directly via JavaScript (for inputs):

```
js.executeScript("arguments[0].value='text'", hiddenInput);
```

3. Change visibility:

```
js.executeScript("arguments[0].style.visibility='visible'", element);
```

4. Trigger click via JavaScript:

```
js.executeScript("arguments[0].click()", element);
```

5. Use `getAttribute()` to read values of hidden elements (this works without visibility):

```
String value = element.getAttribute("value");
```

Note: Interacting with hidden elements may not reflect real user behavior.

Q86.

[Tricky]

What are test design techniques? Explain BVA and ECP with examples.

Test Design Techniques are systematic methods to select test cases efficiently.

1. Equivalence Class Partitioning (ECP):

- Divide input data into equivalent groups (partitions)
- One test per partition is sufficient

Example: Age field (valid: 18-60)

- Partition 1: Invalid below (< 18) → test with 10
- Partition 2: Valid (18-60) → test with 35
- Partition 3: Invalid above (> 60) → test with 70

2. Boundary Value Analysis (BVA):

- Test at the boundaries of equivalence classes
- Errors often occur at boundaries

Example: Age field (valid: 18-60)

- Test values: 17, 18, 19 (lower boundary)
- Test values: 59, 60, 61 (upper boundary)

BVA complements ECP by specifically targeting edge cases at partition boundaries.

Q87.

[Java]

How do you set up a Selenium WebDriver project in Java using Maven?

Step 1: Create a Maven project (pom.xml)

Step 2: Add dependencies to pom.xml:

```
<dependencies>
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>4.18.1</version>
</dependency>
<dependency>
<groupId>org.testng</groupId>
<artifactId>testng</artifactId>
<version>7.9.0</version>
<scope>test</scope>
```

```
</dependency>
<dependency>
<groupId>io.github.bonigarcia</groupId>
<artifactId>webdrivermanager</artifactId>
<version>5.7.0</version>
</dependency>
</dependencies>
```

Step 3: Create test class:

```
public class MyTest {
    WebDriver driver;

    @BeforeMethod
    public void setUp() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
    }

    @Test
    public void testGoogle() {
        driver.get("https://google.com");
        Assert.assertEquals(driver.getTitle(), "Google");
    }

    @AfterMethod
    public void tearDown() { driver.quit(); }
}
```

Q88.

[Java]

How do you read and write test data from Excel files using Apache POI?

Add dependency:

```
<dependency>
<groupId>org.apache.poi</groupId>
<artifactId>poi-ooxml</artifactId>
<version>5.2.5</version>
</dependency>
```

Reading from Excel:

```
FileInputStream fis = new FileInputStream("testdata.xlsx");
Workbook wb = new XSSFWorkbook(fis);
Sheet sheet = wb.getSheet("Sheet1");

Row row = sheet.getRow(1); // Row index (0-based)
Cell cell = row.getCell(0); // Column index

String value = cell.getStringCellValue();
wb.close();
```

Writing to Excel:

```
Workbook wb = new XSSFWorkbook();
Sheet sheet = wb.createSheet("Results");
Row row = sheet.createRow(0);
Cell cell = row.createCell(0);
cell.setCellValue("Test Result: PASS");
```

```
FileOutputStream fos = new FileOutputStream("results.xlsx");
wb.write(fos);
wb.close();
```

Q89.

[Java]

How do you implement the Singleton WebDriver pattern in a Java Selenium framework?

The Singleton pattern ensures only one WebDriver instance exists per thread.

Thread-safe Singleton using ThreadLocal:

```
public class DriverManager {
    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();
```

```
    public static WebDriver getDriver() {
        if (driver.get() == null) {
            initDriver("chrome");
        }
```

```
        return driver.get();
    }
```

```
    public static void initDriver(String browser) {
        WebDriver webDriver;
        switch (browser.toLowerCase()) {
```

```
        case "firefox":
```

```
            webDriver = new FirefoxDriver();
            break;
```

```
        default:
```

```
            webDriver = new ChromeDriver();
        }
```

```
        webDriver.manage().window().maximize();
        driver.set(webDriver);
```

```
    }
```

```
    public static void quitDriver() {
        if (driver.get() != null) {
            driver.get().quit();
            driver.remove();
```

```
        }
```

```
    }
```

```
}
```

```
// Usage in tests:  
WebDriver driver = DriverManager.getDriver();
```

Q90.

[Java]

What are common exceptions in Selenium WebDriver Java? How do you handle them?

Common Exceptions and Handling:

1. NoSuchElementException:

```
try {  
    driver.findElement(By.id("btn")).click();  
} catch (NoSuchElementException e) {  
    System.out.println("Element not found: " + e.getMessage());  
}
```

2. TimeoutException:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
try {  
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("el")));  
} catch (TimeoutException e) {  
    System.out.println("Element not visible within timeout");  
}
```

3. StaleElementReferenceException:

```
for (int i = 0; i < 3; i++) {  
    try {  
        driver.findElement(By.id("el")).click(); break;  
    } catch (StaleElementReferenceException e) { /* retry */ }  
}
```

4. ElementNotInteractableException:

```
JavascriptExecutor js = (JavascriptExecutor) driver;  
js.executeScript("arguments[0].click()", element);
```

Q91.

[Java]

How do you use Java Collections and Streams API with Selenium WebElement lists?

Java Streams with WebElement lists:

```
// Get all links and print their text  
List<WebElement> links = driver.findElements(By.tagName("a"));
```

```
// Filter visible links using Streams
List<WebElement> visibleLinks = links.stream()
    .filter(WebElement::isDisplayed)
    .collect(Collectors.toList());

// Extract all href attributes
List<String> urls = links.stream()
    .map(e -> e.getAttribute("href"))
    .filter(Objects::nonNull)
    .collect(Collectors.toList());

// Check if any link contains text
boolean hasAbout = links.stream()
    .anyMatch(e -> e.getText().contains("About"));

// Count enabled checkboxes
List<WebElement> checkboxes =
    driver.findElements(By.cssSelector("input[type='checkbox']"));

long enabledCount = checkboxes.stream()
    .filter(WebElement::isEnabled)
    .count();

// Sort elements by text
List<String> sortedTexts = links.stream()
    .map(WebElement::getText)
    .sorted()
    .collect(Collectors.toList());
```

Q92.

[Java]

How do you implement method chaining / Fluent Interface pattern in a Java Selenium framework?

The Fluent Interface pattern allows chaining method calls for more readable test code. Each method returns the current page object (this).

LoginPage with fluent interface:

```
public class LoginPage {
    private WebDriver driver;

    @FindBy(id = "username") WebElement usernameField;
    @FindBy(id = "password") WebElement passwordField;
    @FindBy(id = "loginBtn") WebElement loginBtn;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
}
```

```

public LoginPage enterUsername(String username) {
    usernameField.sendKeys(username);

    return this; // Return this for chaining
}

public LoginPage enterPassword(String password) {
    passwordField.sendKeys(password);
    return this;
}

public DashboardPage clickLogin() {
    loginBtn.click();

    return new DashboardPage(driver); // Navigate to next page
}
}

// Usage in test:

new LoginPage(driver)
    .enterUsername("admin")
    .enterPassword("password")
    .clickLogin()

    .verifyDashboardLoaded();

```

Q93.

[Python]

How do you set up Selenium WebDriver with Python?

Step 1: Install Selenium

```
pip install selenium
```

Step 2: Install a browser driver

- Chrome: pip install webdriver-manager (or download ChromeDriver manually)
- Selenium 4.6+: Selenium Manager handles drivers automatically

Step 3: Write your first test

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
# Selenium 4+ (automatic driver management)
```

```
driver = webdriver.Chrome() # No manual driver needed
```

```
driver.get("https://www.google.com")
```

```
print(driver.title)
```

```
driver.quit()
```

```
# With WebDriverManager (older approach):
```

```
from webdriver_manager.chrome import ChromeDriverManager
```

```
from selenium.webdriver.chrome.service import Service
```

```
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)

driver.get("https://www.google.com")
driver.quit()
```

Q94.

[Python]

How do you use the pytest framework with Selenium in Python?

Install:

```
pip install selenium pytest pytest-html
```

Basic pytest structure:

```
import pytest

from selenium import webdriver
from selenium.webdriver.common.by import By

@pytest.fixture(scope="function")
def driver():

    driver = webdriver.Chrome()

    driver.maximize_window()

    yield driver # Provide driver to test

    driver.quit() # Teardown after test


def test_google_title(driver):
    driver.get("https://www.google.com")
    assert "Google" in driver.title


def test_search(driver):
    driver.get("https://www.google.com")

    search_box = driver.find_element(By.NAME, "q")
    search_box.send_keys("Selenium Python")
    search_box.submit()
    assert "Selenium Python" in driver.title


# Run tests:
# pytest test_google.py -v
# pytest test_google.py --html=report.html
```

Q95.

[Python]

How do you implement headless browser testing with Python Selenium?

Headless testing runs browsers without a visible UI – ideal for CI/CD.

Chrome Headless:

```

from selenium import webdriver
from selenium.webdriver.chrome.options import Options

chrome_options = Options()
chrome_options.add_argument("--headless")
chrome_options.add_argument("--no-sandbox")
chrome_options.add_argument("--disable-dev-shm-usage")
chrome_options.add_argument("--window-size=1920,1080")

driver = webdriver.Chrome(options=chrome_options)

driver.get("https://www.google.com")

print(driver.title)

driver.save_screenshot("headless_screenshot.png")
driver.quit()

```

Firefox Headless:

```

from selenium.webdriver.firefox.options import Options as FirefoxOptions

firefox_options = FirefoxOptions()
firefox_options.add_argument("--headless")
driver = webdriver.Firefox(options=firefox_options)

```

Edge Headless:

```

from selenium.webdriver.edge.options import Options as EdgeOptions

edge_options = EdgeOptions()
edge_options.add_argument("--headless")
driver = webdriver.Edge(options=edge_options)

```

Q96.

[Python]

How do you handle dynamic web elements and waits in Python Selenium?

Explicit Wait with Expected Conditions:

```

from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
from selenium import webdriver

```

```

driver = webdriver.Chrome()

```

```

driver.get("https://example.com")

```

```

wait = WebDriverWait(driver, 10)

```

```

# Wait for element to be visible

```

```

element = wait.until(EC.visibility_of_element_located((By.ID, "dynamic-element")))

```

```

# Wait for element to be clickable
button = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[@id='submit']")))
button.click()

# Wait for text to be present
wait.until(EC.text_to_be_present_in_element((By.ID, "status"), "Loaded"))

# Custom wait condition
wait.until(lambda driver: driver.find_element(By.ID, "el").is_displayed())

# Fluent Wait equivalent:
from selenium.webdriver.support.wait import WebDriverWait
from selenium.common.exceptions import NoSuchElementException

wait = WebDriverWait(driver, 10, poll_frequency=0.5,
ignored_exceptions=[NoSuchElementException])

```

Q97.

[Python]

How do you perform parallel test execution in Python Selenium using pytest-xdist?

Install:

```
pip install pytest-xdist pytest-html selenium
```

conftest.py (thread-safe fixtures):

```

import pytest

from selenium import webdriver
from selenium.webdriver.chrome.options import Options

@pytest.fixture(scope="function")
def driver(request):
    options = Options()
    options.add_argument("--headless")
    options.add_argument("--no-sandbox")
    driver = webdriver.Chrome(options=options)

    driver.maximize_window()

    yield driver

    driver.quit()

```

Test file:

```

def test_login(driver):
    driver.get("https://example.com/login")

```

```
# ... test logic

def test_search(driver):
    driver.get("https://example.com")

# ... test logic

Run in parallel:
# Run with 4 parallel workers:

pytest tests/ -n 4

# Run with auto-detect CPU count:

pytest tests/ -n auto

# Combine with HTML report:

pytest tests/ -n 4 --html=report.html
```

Q98.

[Python]

How do you take screenshots and generate HTML test reports in Python Selenium?

Screenshots:

```
import os

from selenium import webdriver
from datetime import datetime

driver = webdriver.Chrome()

# Basic screenshot:

driver.save_screenshot("screenshot.png")

# Screenshot with timestamp:
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

driver.save_screenshot(f"screenshots/screenshot_{timestamp}.png")

# Screenshot as Base64:
screenshot_base64 = driver.get_screenshot_as_base64()

# Element screenshot (Selenium 4):
element = driver.find_element(By.ID, "header")
element.screenshot("element.png")

HTML Reports with pytest-html:
# Install: pip install pytest-html
# Run: pytest tests/ --html=reports/report.html --self-contained-html
```

Allure Reports (more advanced):

Install: pip install allure-pytest

Run: pytest tests/ --alluredir=allure-results

Generate: allure serve allure-results

Screenshot on failure (pytest confest.py):

```
@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
```

```
outcome = yield
```

```
report = outcome.get_result()
```

```
if report.when == "call" and report.failed:
```

```
driver = item.funcargs.get("driver")
```

```
if driver:
```

```
driver.save_screenshot(f"screenshots/{item.name}.png")
```

Q99.

[Advanced]

What are the new features introduced in Selenium 4? How is it different from Selenium 3?

Key New Features in Selenium 4:

1. W3C WebDriver Standard – fully compliant; JSON Wire Protocol retired

2. Selenium Manager – automatic browser driver management (no ChromeDriver manual setup)

3. Relative Locators:

```
WebElement password = driver.findElement(
    RelativeLocator.with(By.tagName("input")).below(usernameField));
```

4. Chrome DevTools Protocol (CDP):

```
DevTools devTools = ((ChromeDriver) driver).getDevTools();
devTools.createSession();
```

5. Improved Screenshots – element-level screenshots

6. New Window/Tab API:

```
driver.switchTo().newWindow(WindowType.TAB);
driver.switchTo().newWindow(WindowType.WINDOW);
```

7. Selenium Grid 4 – Docker-based, improved UI, auto-discovery

8. Better iframe handling:

```
driver.switchTo().frame(frameElement);
```

9. BiDi (Bidirectional) support – real-time event handling

Differences from Selenium 3:

- No more Desired Capabilities (use Options instead)
- No separate FirefoxDriver/ChromeDriver setup needed
- Grid 4 is completely rewritten

Q100.

[Advanced]

How do you handle Shadow DOM elements in Selenium?

Shadow DOM encapsulates HTML, CSS, and JS within a component. Regular findElement doesn't work across shadow boundaries.

Selenium 4 approach (native support):

```
// Step 1: Get the shadow host element
WebElement shadowHost = driver.findElement(By.cssSelector("#shadow-host"));
```

```
// Step 2: Get the shadow root
SearchContext shadowRoot = shadowHost.getShadowRoot();
```

```
// Step 3: Find element within shadow DOM
WebElement shadowElement = shadowRoot.findElement(By.cssSelector(".shadow-button"));
shadowElement.click();
```

JavaScript approach (works in Selenium 3 and 4):

```
JavascriptExecutor js = (JavascriptExecutor) driver;
WebElement shadowRoot = (WebElement) js.executeScript(
    "return arguments[0].shadowRoot", shadowHost);
WebElement element = (WebElement) js.executeScript(
    "return arguments[0].querySelector('.shadow-btn')", shadowRoot);
```

Python (Selenium 4):

```
shadow_host = driver.find_element(By.CSS_SELECTOR, "#shadow-host")
shadow_root = shadow_host.shadow_root
element = shadow_root.find_element(By.CSS_SELECTOR, ".shadow-button")
```

Q101.

[Advanced]

What is the Chrome DevTools Protocol (CDP) in Selenium 4? How do you use it for network interception?

Chrome DevTools Protocol (CDP) allows direct communication with Chrome's DevTools, enabling advanced capabilities like network interception, performance monitoring, and more.

Setup:

```
DevTools devTools = ((ChromeDriver) driver).getDevTools();
devTools.createSession();
```

Network Interception (block requests):

```
devTools.send(Network.enable(Optional.empty(), Optional.empty(), Optional.empty()));
```

```
devTools.addListener(Network.requestWillBeSent(), request -> {
    System.out.println("Request: " + request.getRequest().getUrl());
});
```

```
// Block specific URLs
devTools.send(Network.setBlockedURLs(List.of("*.png", "*.jpg")));
```

Simulate slow network:

```
devTools.send(Network.emulateNetworkConditions(
    false, 100, 50000, 50000,
```

```
Optional.of(Network.ConnectionType.CELLULAR3G)));
```

Mock geolocation:

```
Map<String, Object> coords = new HashMap<>();
coords.put("latitude", 37.7749);
coords.put("longitude", -122.4194);
coords.put("accuracy", 1);
driver.executeCdpCommand("Emulation.setGeolocationOverride", coords);
```

Q102.

[Advanced]

What are Relative Locators in Selenium 4? Provide examples for each type.

Relative Locators (also called Friendly Locators) locate elements based on their position relative to other elements. Import: `import static org.openqa.selenium.support.locators.RelativeLocator.*;`

1. `above()` – element directly above another:

```
WebElement emailLabel = driver.findElement(
    with(By.tagName("label")).above(emailInput));
```

2. `below()` – element directly below another:

```
WebElement passwordField = driver.findElement(
    with(By.tagName("input")).below(usernameField));
```

3. `toLeftOf()` – element to the left:

```
WebElement cancelBtn = driver.findElement(
    with(By.tagName("button")).toLeftOf(submitBtn));
```

4. `toRightOf()` – element to the right:

```
WebElement submitBtn = driver.findElement(
with(By.tagName("button")).toRightOf(cancelBtn));
```

5. near() – element within 50 pixels (default) proximity:

```
WebElement helpIcon = driver.findElement(
with(By.tagName("span")).near(passwordField));
```

Combining locators:

```
WebElement element = driver.findElement(
with(By.tagName("input")).below(usernameLabel).toRightOf(icon));
```

Q103.

[Advanced]

What is Selenium Manager? How does it simplify browser driver management?

Selenium Manager is a tool introduced in Selenium 4.6 that automatically manages browser drivers, eliminating the need for manual ChromeDriver/GeckoDriver setup.

Before Selenium Manager:

- Had to manually download ChromeDriver matching Chrome version
- Set system property: `System.setProperty("webdriver.chrome.driver", "/path/chromedriver");`
- Or use WebDriverManager (third-party library)

With Selenium Manager (Selenium 4.6+):

```
// Just create the driver - no setup needed!
WebDriver driver = new ChromeDriver();
// Selenium Manager automatically downloads the right ChromeDriver
```

How it works:

1. Detects the installed browser version
2. Checks the cache for a compatible driver
3. Downloads the appropriate driver if not cached
4. Configures the driver path automatically

Features:

- Supports Chrome, Firefox, Edge
- Works offline with cached drivers
- No PATH configuration needed
- Handles driver-browser version compatibility automatically

Q104.

[Advanced]

How do you handle dynamic web tables in Selenium?

Dynamic tables have content that changes (pagination, AJAX loading, sorting).

Iterate all rows and columns:

```
WebElement table = driver.findElement(By.id("dataTable"));
List<WebElement> rows = table.findElements(By.tagName("tr"));
```

```
for (WebElement row : rows) {
    List<WebElement> cells = row.findElements(By.tagName("td"));
    for (WebElement cell : cells) {
        System.out.print(cell.getText() + " | ");
    }
    System.out.println();
}
```

Get specific cell (row 2, col 3):

```
String cellText = driver.findElement(
    By.xpath("//table[@id='dataTable']/tbody/tr[2]/td[3]")).getText();
```

Search for a specific value:

```
List<WebElement> cells = driver.findElements(By.xpath("//td"));
for (WebElement cell : cells) {
    if (cell.getText().equals("John")) {
        System.out.println("Found John!");
        break;
    }
}
```

Handle pagination:

```
while (true) {
    // Process current page
    WebElement nextBtn = driver.findElement(By.id("nextPage"));
    if (!nextBtn.isEnabled()) break;
    nextBtn.click();
}
```

Q105.

[Automation]

What are the different types of test automation frameworks used with Selenium?

1. Linear (Record & Playback):

- Simple sequential scripts
- No reusability; good for quick demos

2. Modular Framework:

- Test cases divided into small, reusable modules
- Each module is independent

3. Data-Driven Framework:

- Test data stored externally (Excel, CSV, DB)
- Same script runs with multiple data sets using @DataProvider (TestNG)

4. Keyword-Driven Framework:

- Actions represented as keywords in Excel
- Non-technical testers can create tests

5. Hybrid Framework:

- Combines data-driven + keyword-driven
- Most enterprise frameworks are hybrid

6. Page Object Model (POM):

- Each page represented as a Java class
- Most popular for Selenium projects

7. BDD Framework (Cucumber/SpecFlow):

- Tests written in Gherkin (Given-When-Then)
- Bridges gap between business and technical teams

8. TDD (Test-Driven Development):

- Tests written before code
- JUnit/TestNG-based

Q106.

[Automation]

What is BDD (Behavior-Driven Development)? How does Cucumber integrate with Selenium?

BDD is a development approach where test scenarios are written in plain English (Gherkin language) that both technical and non-technical stakeholders can understand.

Gherkin syntax:

Feature: User Login

Scenario: Valid login

Given the user is on the login page

When the user enters username "admin" and password "pass"

Then the user should be redirected to the dashboard

Cucumber-Selenium Integration:

Step 1: Add dependencies (Maven):

```
<dependency>io.cucumber:cucumber-java</dependency>
```

```
<dependency>io.cucumber:cucumber-testng</dependency>
```

Step 2: Create Step Definitions:

```

@Given("the user is on the login page")
public void openLoginPage() {
driver.get("https://example.com/login");
}

@When("the user enters username {string} and password {string}")
public void enterCredentials(String user, String pass) {
driver.findElement(By.id("username")).sendKeys(user);
driver.findElement(By.id("password")).sendKeys(pass);
driver.findElement(By.id("loginBtn")).click();
}

@Then("the user should be redirected to the dashboard")
public void verifyDashboard() {
Assert.assertTrue(driver.getTitle().contains("Dashboard"));
}

```

Q107.

[Automation]

How do you design a Hybrid Automation Framework in Selenium?

A Hybrid Framework combines Data-Driven and Keyword-Driven approaches.

Structure:

```

project/
src/
main/java/
pages/ # Page Object classes
keywords/ # Keyword action methods
utils/ # Excel reader, WebDriver utils
test/java/
tests/ # Test classes
runners/ # TestNG runner
testdata/ # Excel data files
config/ # config.properties
reports/ # Test reports

```

Key components:

1. Config Manager:

```

public class ConfigManager {
Properties props = new Properties();
// Load browser, URL, timeout from config.properties
}

```

2. ExcelUtil (Data-Driven):

```

public class ExcelUtil {
public static String getCellData(String sheet, int row, int col) { }
public static void setCellData(String sheet, int row, int col, String data) { }
}

```

```
}
```

3. Page Objects (POM):

```
public class LoginPage { @FindBy elements + methods }
```

4. Keyword Engine:

```
public class Keywords {  
    public static void click(WebElement el) { el.click(); }  
    public static void type(WebElement el, String text) { el.sendKeys(text); }  
}
```

5. TestNG with DataProvider combining all layers

Q108.

[Automation]

How do you generate Extent Reports in Selenium?

ExtentReports is a popular HTML reporting library for Selenium.

Step 1: Add Maven dependency:

```
<dependency>  
<groupId>com.aventstack</groupId>  
<artifactId>extentreports</artifactId>  
<version>5.1.1</version>  
</dependency>
```

Step 2: Setup in Base class:

```
public class BaseTest {  
    protected static ExtentReports extent;  
    protected static ExtentTest test;
```

@BeforeSuite

```
public static void setupReport() {  
    ExtentSparkReporter spark = new ExtentSparkReporter("reports/extent.html");  
    spark.config().setReportName("Selenium Test Report");  
    extent = new ExtentReports();  
    extent.attachReporter(spark);  
}
```

@BeforeMethod

```
public void startTest(Method method) {  
    test = extent.createTest(method.getName());
```

```
}
```

@AfterMethod

```
public void endTest(ITestResult result) {  
    if (result.getStatus() == ITestResult.FAILURE) {  
        test.fail(result.getThrowable());  
        // Attach screenshot  
    } else {
```

```
test.pass("Test passed");
}
}

@AfterSuite
public static void flushReport() { extent.flush(); }
}
```

Q109.

[Automation]

How do you integrate Selenium tests into a CI/CD pipeline using Jenkins?

Step 1: Setup Jenkins

- Install Jenkins (local or cloud)
- Install plugins: Git, Maven, TestNG, HTML Publisher

Step 2: Configure Maven project

- Ensure pom.xml has selenium + testng dependencies
- Add maven-surefire-plugin for running tests:

```
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<configuration>
<suiteXmlFiles>
<suiteXmlFile>testng.xml</suiteXmlFile>
</suiteXmlFiles>
</configuration>
</plugin>
```

Step 3: Create Jenkins Job (Freestyle or Pipeline)

- Source Code: Git repo URL + credentials
- Build Trigger: GitHub webhook (on push) or cron schedule
- Build Step: Execute Maven goals: mvn clean test

Step 4: Publish Reports

- Add Post-build action: Publish HTML Reports
- HTML directory: target/surefire-reports
- Or use Allure plugin for better reports

Step 5: Notifications

- Add email notifications on failure
- Integrate Slack notifications via Slack plugin

Q110.

[Automation]

What is cross-browser testing with Selenium Grid and Docker? How do you set it up?

Selenium Grid 4 with Docker Compose:

Step 1: Create docker-compose.yml

version: '3'

services:

selenium-hub:

image: selenium/hub:4

ports:

- "4444:4444"

chrome:

image: selenium/node-chrome:4

environment:

- SE_EVENT_BUS_HOST=selenium-hub

- SE_EVENT_BUS_PUBLISH_PORT=4442

- SE_EVENT_BUS_SUBSCRIBE_PORT=4443

depends_on:

- selenium-hub

firefox:

image: selenium/node-firefox:4

environment:

- SE_EVENT_BUS_HOST=selenium-hub

- SE_EVENT_BUS_PUBLISH_PORT=4442

- SE_EVENT_BUS_SUBSCRIBE_PORT=4443

depends_on:

- selenium-hub

Step 2: Start Grid

```
docker-compose up -d
```

Step 3: Run tests on Grid

```
ChromeOptions options = new ChromeOptions();
WebDriver driver = new RemoteWebDriver(
    new URL("http://localhost:4444"), options);
```

Step 4: View Grid at <http://localhost:4444/ui>

Q111.

[MCQ]

MCQ 1: What is the utility of a Selenium Web Driver?

Answer: C) It drives the web browser

Selenium WebDriver is used to automate and drive web browsers. It provides a programming interface to control browser actions like navigating to URLs, clicking elements, filling forms, and verifying page content.

Options typically given:

- A) It is used to test mobile applications
- B) It is used to manage databases
- C) It drives the web browser ✓
- D) It is used for load testing

Q112.

[MCQ]

MCQ 2: Which Selenium command checks if a certain piece of text is present on the page?

Answer: B) verifyTextPresent

The verifyTextPresent command (in Selenium IDE/RC) checks whether the specified text appears anywhere on the current page. If not found, it logs a failure but continues execution (unlike assertTextPresent which stops the test).

Options typically given:

- A) assertText
- B) verifyTextPresent ✓
- C) checkText
- D) findText

Q113.

[MCQ]

MCQ 3: Which language is supported by Selenium Web Driver?

Answer: D) All of the above (Java, Python, C#, Ruby, JavaScript)

Selenium WebDriver officially supports multiple programming languages:

- Java
- Python
- C# (.NET)
- Ruby
- JavaScript (Node.js)
- Kotlin (via Java bindings)

This multi-language support is one of Selenium's major advantages over competing tools.

Q114.

[MCQ]

MCQ 4: Which Java-based build tool might be needed for Selenium Grid?

Answer: B) Maven

Maven is the most commonly used Java build tool for Selenium Grid projects. It manages dependencies (selenium-java, testng, etc.) through pom.xml and provides commands like mvn clean test to execute test suites.

Gradle is also a valid build tool, but Maven is more widely associated with traditional Selenium Grid setups.

Options typically given:

- A) Ant
- B) Maven ✓
- C) Gradle
- D) Make

Q115.

[MCQ]

MCQ 5: Which of the following is true about Action commands in Selenium?

Answer: B) Action commands manipulate the state of the application

Action commands (in Selenium IDE/Selenese) directly interact with and manipulate the application state — they perform user actions such as clicking, typing, selecting, dragging, etc.

Key fact: Action commands ARE the ones that interact with the UI. Accessors READ values, and Assertions VERIFY states.

Q116.

[MCQ]

MCQ 6: Which of the following is NOT a Selenium component?

Answer: C) Selenium Studio (or whichever option is not part of the suite)

The four official Selenium components are:

- Selenium IDE ✓
- Selenium RC (Remote Control) ✓
- Selenium WebDriver ✓
- Selenium Grid ✓

Common wrong answer options include: "Selenium Studio", "Selenium Builder", or "Selenium Server" (which is part of Grid, not a separate component).

Q117.

[MCQ]

MCQ 7: Which is NOT true about Selenium with Globs?

Answer: The option stating Globs support regular expressions is NOT true.

Globs in Selenium IDE are a simple pattern matching mechanism:

- * matches any sequence of characters
- ? matches any single character
- They do NOT support full regular expressions
- For regex, use the regexp: prefix in Selenium IDE
- Globbs cannot match patterns like \d+, [a-z], etc. (regex syntax)

The distinction: Globbs = simple wildcards; Regex = full pattern matching

Q118.

[MCQ]

MCQ 8: Which is faster: Selenium RC or Selenium WebDriver?

Answer: B) Selenium WebDriver

Selenium WebDriver is significantly faster than Selenium RC because:

- WebDriver communicates directly with the browser via native browser support
- Selenium RC required an intermediate server (RC Server) that received commands, then injected JavaScript into the browser
- The extra server hop in RC added latency
- WebDriver uses browser-native automation APIs, eliminating the proxy server

This is one of the main reasons Selenium RC was deprecated in favor of WebDriver.

Q119.

[MCQ]

MCQ 9: Which facility is NOT supported by Selenium Web Driver?

Answer: C) Testing desktop applications (or database testing, etc.)

Selenium WebDriver does NOT support:

- Desktop application testing
- Direct database testing
- Testing non-browser-based applications
- Native mobile app testing
- Image comparison/visual testing natively
- CAPTCHA handling

Selenium WebDriver ONLY supports web browser automation. For desktop apps, tools like AutoIT, WinAppDriver, or SikuliX are needed.

Q120.

[MCQ]

MCQ 10: Adding 'waitFor' commands when tests lack required items is related to which of the following?

Answer: B) Debugging / Test Synchronization

Adding `waitFor` commands is a debugging and synchronization technique used when:

- Tests fail due to elements not being ready
- Asynchronous content (AJAX) hasn't loaded
- The test is executing faster than the application responds

`waitFor` commands (like `waitForElementVisible`, `waitForTextPresent`) pause execution until a condition is met, helping tests synchronize with the application state. This is preferable to hard-coded sleeps.

Q121.

[MCQ]

MCQ 11: Which two instructions best confirm that a button has appeared on a page?

Answer: `assertElementPresent` and `assertVisible` (or `verifyElementPresent` and `verifyVisible`)

To confirm a button has appeared:

1. `assertElementPresent(locator)` - confirms the element exists in the DOM
2. `assertVisible(locator)` - confirms the element is visible to the user

Using both together provides complete confirmation:

- Element could be in DOM but hidden (`display:none`) — so presence alone isn't enough
- `assertVisible` ensures the element is actually displayed on screen
- Together they confirm the button is both present AND visible

Q122.

[MCQ]

MCQ 12: What does `assertTitle` look for?

Answer: B) The title of the current page

`assertTitle(expectedTitle)` verifies that the current browser page title exactly matches the expected title.

Example in Selenium IDE:

Command: `assertTitle`

Value: "Google"

→ Passes if the page title is exactly "Google"

→ Fails and stops execution if title doesn't match

It is equivalent to:

```
Assert.assertEquals(driver.getTitle(), "Google"); // Java
```

Q123.

[MCQ]

MCQ 13: Which variation finds elements in Selenium WebDriver using an underlying CSS generator?

Answer: B) CSS Selector

CSS Selector uses CSS syntax to locate elements and is implemented using the browser's built-in CSS engine (CSS generator/parser).

Examples:

```
driver.findElement(By.cssSelector("#username")); // by ID
driver.findElement(By.cssSelector(".btn-primary")); // by class
driver.findElement(By.cssSelector("input[type='text']")); // by attribute
```

CSS Selectors are often faster than XPath in Chrome and Firefox because browsers are optimized for CSS parsing. They use the underlying browser CSS engine to match elements.

Q124.

[MCQ]

MCQ 14: Is SelectAllOptions() a valid command in WebDriver?

Answer: B) No

SelectAllOptions() is NOT a valid command in Selenium WebDriver.

For selecting options in a dropdown, Selenium provides the Select class:

```
Select select = new Select(driver.findElement(By.id("dropdown")));
```

Valid Select methods:

- selectByVisibleText("Option 1")
- selectByValue("val1")
- selectByIndex(0)
- deselectAll() – deselects all options (multi-select only)
- getOptions() – returns all options
- getAllSelectedOptions() – returns selected options

To select ALL options (if multi-select), you need to loop through options manually.

Q125.

[MCQ]

MCQ 15: In WebDriver, which command can be used to enter values in a text box?

Answer: B) sendKeys()

The sendKeys() method is the standard way to enter text into a text box in Selenium WebDriver.

Usage:

```
driver.findElement(By.id("username")).sendKeys("admin");
```

Other related methods:

- `clear()` – clears existing text before typing
- `sendKeys(Keys.ENTER)` – sends keyboard keys

Alternative (non-standard):

- `JavaScriptExecutor` to set value directly
- `Actions` class with `sendKeys`

Options typically given:

A) `type()` B) `sendKeys()` ✓ C) `input()` D) `enterText()`

Q126.

[MCQ]

MCQ 16: In Selenium, which of the following is NOT a wait command?

Answer: D) `staticWait()` (or similar made-up option)

Valid wait mechanisms in Selenium:

- `implicitlyWait()` ✓ – global wait for element detection
- `WebDriverWait` (Explicit Wait) ✓ – wait for specific condition
- `FluentWait` ✓ – customizable polling wait
- `pageLoadTimeout()` ✓ – wait for page load
- `Thread.sleep()` ✓ – Java static wait (not recommended)

NOT a valid Selenium wait:

- `staticWait()`, `hardWait()`, `pauseWait()` – these don't exist in WebDriver API

Q127.

[MCQ]

MCQ 17: How to locate an element by its ID in Selenium Web Driver?

Answer: B) `driver.findElement(By.id("elementId"))`

The correct syntax to locate an element by ID:

```
WebElement element = driver.findElement(By.id("username"));
```

ID is the fastest and most reliable locator because:

- IDs should be unique on a page
- Browsers are optimized for ID-based lookup
- Less fragile than XPath or CSS

Python equivalent:

```
element = driver.find_element(By.ID, "username")
```

Q128.

[MCQ]

MCQ 18: How does the WebDriver window get closed?

Answer: Both `driver.close()` and `driver.quit()` can close WebDriver windows, but they differ:

`driver.close()`:

- Closes the currently active/focused browser window
- If multiple windows are open, only the current one closes
- Session remains active

`driver.quit()`:

- Closes ALL browser windows opened by the WebDriver session
- Ends the WebDriver session completely
- Releases all resources

Best practice: Always use `driver.quit()` at the end of tests to prevent memory leaks and orphaned browser processes.

Q129.

[MCQ]

MCQ 19: How can you execute a certain command in Selenium IDE?

Answer: C) By clicking the Run button or using the Step feature

In Selenium IDE, you can execute commands by:

- Run Test (play button) – runs the entire test
- Step button – executes one command at a time (useful for debugging)
- Run Current Command – executes only the selected/highlighted command
- Keyboard shortcut: X key to execute a single command

The Step feature is particularly useful for troubleshooting because you can:

- Pause execution at any step
- Inspect the application state
- Execute one command at a time to find the failing step

Q130.

[MCQ]

MCQ 20: Choose a language that Selenium RC does NOT support.

Answer: The language NOT supported by Selenium RC (among common options) is typically PHP or a newer language like Kotlin/Swift.

Languages Selenium RC DID support:

- Java ✓
- Python ✓
- C# ✓
- Ruby ✓
- Perl ✓
- PHP ✓ (basic support)

Note: Selenium RC is deprecated. Selenium WebDriver supports Java, Python, C#, Ruby, and JavaScript (Node.js) officially. If the question asks about a specific option like "Kotlin" or "Swift" — those were not natively supported by Selenium RC.

Q131.

[GFG]

What is the difference between Selenium 2.0 and Selenium 3.0?

Selenium 2.0:

- Combined the original Selenium project with the WebDriver project
- Introduced Selenium WebDriver as the primary automation interface
- Selenium RC (Remote Control) became outdated after the merge, but was still supported to avoid breaking existing tests
- Provided a unified API for browser automation

Selenium 3.0:

- Built upon Selenium 2.0 with a range of improvements
- Completely removed reliance on Selenium RC
- Seamlessly worked with older tests (backward compatible)
- Fixed numerous bugs and improved overall reliability
- Improved Firefox support via GeckoDriver (W3C standard)
- Better support for Edge, Safari browsers

In short, Selenium 3.0 took the strengths of Selenium 2.0, removed deprecated components, and made it more stable and reliable for automating web tests.

Q132.

[GFG]

What are the testing types supported by Selenium?

Selenium primarily supports two major types of testing:

1. Regression Testing:

- Tests modified parts of the code and areas that might be affected by changes
- Ensures no new errors are introduced after modifications
- Steps involved:
 - Identify relevant test cases from existing suites
 - Prioritize based on criticality and priority
 - Execute selected test cases to detect regressions
 - Analyze results for failures
 - Report defects in bug tracking system
 - Automate regression tests for efficiency
 - Repeat after each software update

2. Functional Testing:

- Verifies that each function of the application works per requirements
- Steps involved:

- Identify test input / functionality to be tested
- Compute expected outcomes based on specifications
- Execute test cases and record output
- Compare actual vs expected output to find deviations

Other testing types supported indirectly: Cross-browser testing, smoke testing, sanity testing, data-driven testing.

Q133.

[GFG]

What are the different types of annotations used in Selenium (TestNG)?

TestNG Annotations used in Selenium:

```
@BeforeSuite - Executed before all test cases in a TestNG Suite run
@AfterSuite - Executed after all test cases in a TestNG Suite complete
@BeforeTest - Executed before all @Test annotated methods in a TestNG Suite
@AfterTest - Executed after all @Test annotated methods in a TestNG Suite
@BeforeClass - Executed before all methods in the current class start
@AfterClass - Executed after all methods in the current class finish
@BeforeMethod - Executed before each test method runs
@AfterMethod - Executed after each test method runs
@BeforeGroups - Executed before all methods of the specified group run
@AfterGroups - Executed after all test methods belonging to a specified group have run
@Test - Marks a method as a test method
@DataProvider - Provides test data for parameterized tests
@Parameters - Passes parameters from testng.xml to test methods
@Listeners - Specifies listener classes for the test
```

Example:

```
@BeforeMethod
public void setUp() {
    driver = new ChromeDriver();
}

@Test
public void testLogin() {
    driver.get("https://example.com");
}

@AfterMethod
public void tearDown() {
    driver.quit();
}
```

Q134.

[GFG]

What are the types of web locating strategies in Selenium?

Selenium provides 7 locating strategies through the By class:

1. By ID:

```
element = driver.find_element(By.ID, "element_id")
```

- Fastest and most reliable; IDs should be unique per page

2. By Class Name:

```
element = driver.find_element(By.CLASS_NAME, "element_class_name")
```

- Multiple elements can share a class; returns first match

3. By Name:

```
element = driver.find_element(By.NAME, "element_name")
```

- Commonly used for form input fields

4. By Tag Name:

```
element = driver.find_element(By.TAG_NAME, "input")
```

- Locates by HTML tag; returns first matching tag

5. By CSS Selector:

```
driver.find_element(By.CSS_SELECTOR, "input#username")
```

- Powerful pattern-based selection; faster than XPath in most browsers

6. By XPath:

```
element = driver.find_element(By.XPATH, "//input[@id='username']")
```

- Most flexible; can navigate complex DOM structures

7. By Link Text / Partial Link Text:

```
element = driver.find_element(By.LINK_TEXT, "Click Here")
```

```
element = driver.find_element(By.PARTIAL_LINK_TEXT, "Click")
```

- Used specifically for anchor <a> elements / hyperlinks

Q135.

[GFG]

What is the difference between Manual Testing and Automated Testing?

Comparison of Manual Testing vs Automation Testing:

Definition:

- Manual Testing – Test cases are executed by human testers
- Automation Testing – Test cases are executed by software tools (like Selenium)

Processing Time:

- Manual Testing – Time-consuming; each test case needs human execution
- Automation Testing – Faster; scripts run automatically without human intervention

Resources Requirement:

- Manual Testing – Requires human resources (testers)

- Automation Testing – Requires automation tools and trained employees

Exploratory Testing:

- Manual Testing – Possible; humans can explore freely
- Automation Testing – Not possible; scripts follow fixed paths

Framework Requirement:

- Manual Testing – No frameworks needed
- Automation Testing – Uses frameworks like Data-Driven, Keyword-Driven, Hybrid, POM

Reliability:

- Manual Testing – Prone to human error
- Automation Testing – More reliable and consistent

Reusability:

- Manual Testing – Test cases cannot be reused automatically
- Automation Testing – Scripts are reusable across builds and environments

Best for:

- Manual Testing – Exploratory, ad-hoc, usability testing
- Automation Testing – Regression, smoke, performance, repeated test cycles

Q136.

[GFG]

What are the limitations of Selenium testing?

Limitations of Selenium:

1. No Dedicated Support:

- Selenium is open-source, so there is no official tech support
- Users rely on community forums and documentation

2. Web Applications Only:

- Selenium is designed for web browser automation only
- Testing mobile or desktop apps requires additional tools (Appium, WinAppDriver)

3. No Image Testing:

- Selenium does not support visual/image comparison directly
- Third-party tools like Applitools or AShot are needed

4. No Built-in Reports or Management:

- Requires integration with TestNG, JUnit, Allure, or Extent Reports

5. Programming Knowledge Needed:

- Basic programming skills are required to write and maintain test scripts

6. Cross-Browser Compatibility Issues:

- Browsers interpret HTML/CSS differently, causing inconsistent results

7. Slow Test Execution:

- Browser drivers add overhead, especially on large web applications

8. Difficulty Handling Dynamic Web Elements:

- Elements with dynamic IDs or attributes can cause script failures

9. Limited Windows-Based Application Support:

- Cannot automate desktop apps; needs third-party tools like AutoIT

10. No CAPTCHA Automation:

- CAPTCHA is intentionally designed to prevent bot interactions

Q137.

[GFG]

What makes Selenium a widely used testing tool?

Reasons Selenium is widely used:

1. Easy to Use:

- Developed with JavaScript underpinnings, accessible across platforms
- Rich documentation and community support

2. Browser Compatibility:

- Supports all major browsers: Chrome, Firefox, Edge, Safari, Opera

3. Language Flexibility:

- Test scripts can be written in Java, Python, C#, Ruby, JavaScript, and more

4. Platform Independent:

- Works on Windows, Linux, and macOS

5. Third-Party Integration:

- Integrates with JUnit, TestNG, Maven, Jenkins, Docker, and more

6. Open-Source and Free:

- No licensing cost; accessible to developers and testers of all levels

7. Cross-Platform Support:

- Compatible with multiple OS and browser combinations

8. Large Community:

- Extensive tutorials, plugins, and community-driven support

9. Selenium Grid:

- Enables parallel and distributed test execution

10. Continuous Integration Support:

- Seamlessly integrates with CI/CD pipelines (Jenkins, GitHub Actions, etc.)

Q138.

[GFG]

Why is it advised to select Selenium as a testing tool for web applications?

Reasons to choose Selenium for web application testing:

- Open-Source and Free – No cost; portable and widely accessible
- Cross-Platform Support – Compatible with Linux, Unix, macOS, and Windows
- Browser Compatibility – Works with Chrome, Firefox, Safari, Edge, Opera, Internet Explorer
- Multi-Language Support – Allows scripting in Java, Python, JavaScript, Ruby, Perl, C#, and more
- Mobile Testing – Supports testing on iPhone, Android, and BlackBerry via Appium integration
- Integration with Build Tools – Easily integrates with ANT and Maven for code compilation
- Resource-Efficient – Uses minimal CPU and RAM for script execution
- Parallel Execution – Selenium Grid allows running tests on multiple machines/browsers simultaneously
- Framework Support – Works with TestNG, JUnit, Cucumber, and more
- Community and Ecosystem – Large community, rich plugins, extensive documentation
- Continuous Integration – Easy integration with Jenkins, GitHub Actions, CircleCI

Q139.

[GFG]

What are the Selenium 4 new features and how is it different from other versions?

Selenium 4 Key Features:

1. W3C WebDriver Standard Compliance:

- Fully compliant with W3C WebDriver spec
- JSON Wire Protocol is retired; more stable communication

2. Selenium Manager:

- Automatic browser driver management (no manual ChromeDriver setup)

3. Relative Locators:

- Locate elements based on position relative to other elements
- `above()`, `below()`, `toLeftOf()`, `toRightOf()`, `near()`

4. Chrome DevTools Protocol (CDP):

- Direct integration with Chrome DevTools for network interception, performance testing, etc.

5. Improved Screenshots:

- Element-level screenshots supported natively

6. New Window/Tab API:

```
driver.switchTo().newWindow(WindowType.TAB);  
driver.switchTo().newWindow(WindowType.WINDOW);
```

7. Selenium Grid 4:

- Completely rewritten; Docker support; improved Grid UI
- Supports Kubernetes and cloud platforms

8. BiDi (Bidirectional) API:

- Real-time event listening and interception

9. Better iframe Handling:

- Improved `switchTo().frame()` support

Differences from Selenium 3:

- No Desired Capabilities (use Options classes instead)
- No manual driver setup needed (Selenium Manager)
- Better debugging with CDP integration
- Relative locators are new and unavailable in Selenium 3

Q140.

[GFG]

Is it possible to test APIs or web services using Selenium WebDriver?

No, Selenium WebDriver cannot be used to test APIs or web services.

Reason:

- Selenium WebDriver is specifically designed for browser-based automation
- It interacts with the UI of web applications (clicking, typing, navigating)
- It does not have the capability to send HTTP requests directly or parse API responses

For API/Web Service Testing, use dedicated tools:

- REST Assured (Java) – popular for REST API testing
- Postman – GUI tool for API testing and automation
- RestSharp (.NET) – REST client for C#
- requests (Python) – HTTP library for API calls
- Karate DSL – BDD-style API testing framework
- SoapUI – for SOAP and REST web services

Combining Selenium with API Testing:

- You can use Selenium for UI testing and REST Assured/requests for API validation in the same test framework
- E.g., use API calls to set up test data, then use Selenium to verify the UI reflects those changes

Q141.

[GFG]

How can we move to the nth-child element using XPath?

To select the nth-child element using XPath, use the following expression:

General syntax:

```
(//parent-element/*)[n]
```

Examples:

Select the 2nd row in a table:

```
(//table/tbody/tr)[2]
```

Select the 3rd list item:

```
(//ul/li)[3]
```

Select the 1st input inside a form:

```
(//form//input)[1]
```

Using position() function:

```
//ul/li[position()=3]
```

Select last element:

```
//ul/li[last()]
```

Select second-to-last:

```
//ul/li[last()-1]
```

Select all even rows:

```
//table/tbody/tr[position() mod 2 = 0]
```

Note: XPath indexing is 1-based (starts from 1, not 0), unlike most programming languages.

Q142.

[GFG]

How to scroll down a page using JavaScript in Selenium?

Using JavascriptExecutor to scroll:

General syntax:

```
executeScript("window.scrollTo(x-pixels, y-pixels)");
```

- x-pixels: Horizontal scroll amount (0 for no horizontal scroll)
- y-pixels: Vertical scroll amount (positive = down, negative = up)

Java Example – Scroll down 1000 pixels:

```
JavascriptExecutor js = (JavascriptExecutor) driver;  
driver.get("https://www.example.com/");  
js.executeScript("window.scrollTo(0, 1000)");
```

Scroll to bottom of page:

```
js.executeScript("window.scrollTo(0, document.body.scrollHeight)");
```

Scroll to top of page:

```
js.executeScript("window.scrollTo(0, 0)");
```

Scroll to a specific element:

```
WebElement element = driver.findElement(By.id("footer"));  
js.executeScript("arguments[0].scrollIntoView(true);", element);
```

Scroll horizontally:

```
js.executeScript("window.scrollTo(500, 0)");
```

Python equivalent:

```
js = driver
```

```
driver.execute_script("window.scrollTo(0, 1000)")
```

Q143.

[GFG]

How to assert the title of a webpage in Selenium?

Steps to assert a webpage title:

Step 1: Open the webpage

```
driver.get("https://www.geeksforgeeks.org/");
```

Step 2: Get the actual title

```
String actualTitle = driver.getTitle();
```

Step 3: Set the expected title

```
String expectedTitle = "GeeksforGeeks - A Computer Science Portal for Geeks";
```

Step 4: Compare using if-else

```
if(actualTitle.equalsIgnoreCase(expectedTitle)) {  
    System.out.println("Title Matched");  
} else {  
    System.out.println("Title didn't match");  
}
```

Step 5: Using Assert (TestNG/JUnit)

```
Assert.assertEquals(actualTitle, expectedTitle);
```

Python example:

```
driver.get("https://www.google.com")
```

```
assert "Google" in driver.title, "Title mismatch!"
```

Or with unittest:

```
self.assertEqual(driver.title, "Google")
```

Note: driver.getTitle() (Java) / driver.title (Python) returns the current page's <title> tag value.

Q144.

[GFG]

How to retrieve CSS properties of an element in Selenium?

Using getCssValue() method in Selenium:

Syntax:

```
String value = element.getCssValue("property-name");
```

Java Examples:

```
// Get background color
```

```
String bgColor = driver.findElement(By.id("header")).getCssValue("background-color");  
System.out.println("Background color: " + bgColor);
```

```
// Get font size
```

```
String fontSize = driver.findElement(By.id("title")).getCssValue("font-size");
```

```
// Get font family
```

```
String fontFamily = driver.findElement(By.id("menu")).getCssValue("font-family");
```

```
// Get color
```

```
String color = driver.findElement(By.cssSelector("h1")).getCssValue("color");
```

```
// Get visibility
```

```
String visibility = element.getCssValue("visibility");
```

Python example:

```
element = driver.find_element(By.ID, "header")
```

```
bg_color = element.value_of_css_property("background-color")
```

```
print(bg_color) # Returns value like "rgba(255, 255, 255, 1)"
```

Common CSS properties to retrieve:

- background-color, color, font-size, font-family
- display, visibility, width, height
- margin, padding, border
- text-decoration, font-weight

Q145.

[GFG]

Why do testers choose Selenium over QTP (UFT)?

Comparison: Selenium vs QTP (now called UFT - Unified Functional Testing):

Cost:

- Selenium – Free and open-source
- QTP/UFT – Paid commercial tool (expensive licensing)

Language Support:

- Selenium – Java, Python, C#, Ruby, JavaScript, and more
- QTP/UFT – Primarily VBScript (limited language options)

Browser Support:

- Selenium – All major browsers (Chrome, Firefox, Edge, Safari)
- QTP/UFT – Limited browser support

OS Compatibility:

- Selenium – Windows, Mac, Linux
- QTP/UFT – Windows only

Application Support:

- Selenium – Web applications only
- QTP/UFT – Web, desktop, SAP, and more

Maintenance:

- Selenium – Community-maintained, frequent updates
- QTP/UFT – Vendor (Micro Focus) support

Integration:

- Selenium – Strong integration with Maven, Jenkins, TestNG, Docker
- QTP/UFT – Limited third-party integration

Conclusion: Selenium is preferred for web applications due to its zero cost, language flexibility, cross-platform support, and extensive ecosystem.

Q146.

[GFG]

What are data-driven and keyword-driven frameworks?

1. Data-Driven Framework:

- A type of test automation framework that separates test data from test scripts
- Test data is stored externally in Excel, CSV, XML, or databases
- The same test script runs multiple times with different data sets
- Makes it easy to add new test scenarios without changing code

Example structure:

Username	Password	Expected Result
admin	admin123	Login Success
user1	wrongpass	Login Failed

Benefits:

- One script handles multiple data combinations
- Easy to maintain and extend test coverage
- Non-technical testers can update test data

2. Keyword-Driven Framework:

- Test actions are represented as keywords stored in external files (usually Excel)
- A keyword engine reads and executes the keywords
- Non-technical users can create test cases without writing code

Example keywords:

Action	Locator	Value
OpenBrowser	Chrome	
NavigateTo		https://x
EnterText	id=username	admin
Click	id=loginBtn	

Benefits:

- Readable by non-technical stakeholders
- Reusable keyword library
- Easy to maintain

Q147.

[GFG]

What is a Selenium Maven project?

A Selenium Maven project is a software project that uses Apache Maven to:

- Manage project dependencies (Selenium libraries, TestNG, etc.)
- Handle the build process (compilation, packaging, test execution)
- Maintain a standardized project structure

Key file – pom.xml (Project Object Model):

Defines all dependencies and build configurations.

Example pom.xml for Selenium:

```
<dependencies>
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>4.18.1</version>
```

```
</dependency>
<dependency>
<groupId>org.testng</groupId>
<artifactId>testng</artifactId>
<version>7.9.0</version>
<scope>test</scope>
</dependency>
</dependencies>
```

Standard Maven project structure:

project/

src/

main/java/ # Page objects, utilities

test/java/ # Test classes

pom.xml # Dependency and build config

Benefits:

- Automatic dependency download
- Consistent build across machines
- Easy CI/CD integration (mvn clean test)
- No manual JAR file management

Q148.

[GFG]

How to submit a form in Selenium?

Methods to submit a form in Selenium:

Method 1: Using submit() on a form element

```
driver.findElement(By.id("myForm")).submit();
```

Method 2: Click the submit button

```
driver.findElement(By.cssSelector("button[type='submit']")).click();
```

Method 3: Press ENTER key

```
driver.findElement(By.id("inputField")).sendKeys(Keys.ENTER);
```

Method 4: JavaScript

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("document.getElementById('myForm').submit()");
```

Full example with form fields:

```
WebElement firstName = driver.findElement(By.id("FirstName"));
WebElement email = driver.findElement(By.id("Email"));
```

```
WebElement submitBtn = driver.findElement(By.cssSelector("button[type='submit']"));
```

```
firstName.sendKeys("John");  
email.sendKeys("john@example.com");  
submitBtn.click();
```

```
// Handle alert after submission  
Alert alert = driver.switchTo().alert();  
String alertText = alert.getText();  
System.out.println("Alert: " + alertText);  
alert.accept();
```

Python example:

```
driver.find_element(By.ID, "FirstName").send_keys("John")  
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
```

Q149.

[GFG]

How to delete cookies in Selenium?

Method 1: Delete all cookies using deleteAllCookies():

Java:

```
driver.manage().deleteAllCookies();
```

Python:

```
driver.delete_all_cookies()
```

JavaScript:

```
await driver.manage().deleteAllCookies();
```

Method 2: Delete a specific cookie by name:

Java:

```
driver.manage().deleteCookieNamed("cookieName");
```

Python:

```
driver.delete_cookie("cookieName")
```

Method 3: Get all cookies first, then delete:

Java:

```
Set<Cookie> allCookies = driver.manage().getCookies();  
for (Cookie cookie : allCookies) {  
    driver.manage().deleteCookieNamed(cookie.getName());  
}
```

Method 4: Via browser settings page:

```
driver.get("chrome://settings/clearBrowserData");  
// Find and click "Clear Data" button
```

Method 5: Add cookie (for reference):

```
Cookie cookie = new Cookie("name", "value");  
driver.manage().addCookie(cookie);
```

When to use:

- Before each test to ensure a clean session
- After login tests to reset state
- To test cookie-dependent functionality

Q150.

[GFG]

How do you work with frames (iframes) in Selenium?

A frame (iframe) is an HTML document embedded within another HTML document.

Step 1: Identify if a frame exists:

- Right-click the page element → if you see "This Frame", "View Frame Source" – it's a frame
- Look for <iframe> or <frame> tags in page source

Step 2: Switch into the frame:

By index (0-based):

```
driver.switchTo().frame(0); // First frame
```

By name or ID:

```
driver.switchTo().frame("frameName");  
driver.switchTo().frame("frameId");
```

By WebElement:

```
WebElement iframeElement = driver.findElement(By.id("myFrame"));  
driver.switchTo().frame(iframeElement);
```

Step 3: Interact with elements inside the frame:

```
// After switching, work with elements normally  
driver.findElement(By.id("elementInsideFrame")).click();
```

Step 4: Switch back to main content:

```
driver.switchTo().defaultContent(); // Back to main page
```

Step 5: Switch to parent frame (nested frames):

```
driver.switchTo().parentFrame(); // Go one level up
```

Python example:

```
driver.switch_to.frame("iframe_id")
driver.find_element(By.ID, "inner_element").click()
driver.switch_to.default_content()
```

Q151.

[GFG]

What is the importance of Cross-Browser Testing in Selenium?

Cross-Browser Testing verifies that a web application works correctly across different browsers.

Why it is important:

1. Ensures Consistent User Experience:

- Different browsers (Chrome, Firefox, Safari, Edge) use different rendering engines
- The same page may look or behave differently; cross-browser testing catches these issues

2. Reaches a Wider Audience:

- Users access websites from diverse browsers and devices
- Ensuring compatibility prevents alienating users

3. Improves Quality and Reduces Risk:

- Catches browser-specific bugs before they reach production
- Reduces user frustration and potential revenue loss

4. Boosts SEO:

- Search engines consider cross-browser compatibility as a ranking factor

5. Regulatory Compliance:

- Some industries require applications to support specific browsers

Benefits of Selenium for Cross-Browser Testing:

- Automation – Automates repetitive tests across multiple browsers
- Flexibility – Supports multiple languages and frameworks
- Open-Source – Cost-effective and accessible
- Parallel Execution – Selenium Grid runs tests on multiple browsers simultaneously

How to implement:

```
ChromeOptions chromeOpts = new ChromeOptions();
FirefoxOptions ffOpts = new FirefoxOptions();
// Run same tests with different driver instances
```

Q152.

What are the challenges of Parallel Execution in Selenium?

Parallel execution runs multiple tests simultaneously, but brings challenges:

1. Test Dependencies:

- Data isolation – Tests sharing data can interfere with each other
- Order dependencies – Some tests need to run in a specific sequence
- Solution: Design independent, stateless tests; use separate data sets per thread

2. Resource Management:

- Running multiple browser instances strains CPU, memory, and network
- Not all WebDrivers support parallel execution equally
- Solution: Use Selenium Grid, cloud services (BrowserStack, Sauce Labs), or Docker

3. Synchronization Issues:

- Multiple tests accessing shared resources cause race conditions
- Solution: Use `ThreadLocal<WebDriver>` for thread-safe driver instances

4. Test Stability / Flakiness:

- Parallelization can introduce unpredictable interactions between tests
- Solution: Proper wait strategies; isolate test environments

5. Debugging Difficulties:

- Identifying root causes of failures in parallel runs is complex
- Solution: Use robust logging and reporting tools (Allure, Extent Reports)

6. Environment Setup Complexity:

- Managing multiple test environments for parallel runs is time-consuming
- Solution: Use Docker containers or cloud testing grids

`ThreadLocal` example for parallel safety:

```
private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();
public static WebDriver getDriver() { return driver.get(); }
```

Q153.

How to integrate Selenium with Jenkins for Continuous Integration?

Step 1: Setting Up Jenkins:

- Install Jenkins (local or on a server)
- Install required plugins: Pipeline, Git, Maven Integration, HTML Publisher, Allure

Step 2: Prepare Your Selenium Tests:

- Organize tests into modules/classes
- Use TestNG or JUnit as the testing framework
- Manage dependencies via Maven (pom.xml)

Step 3: Create a Jenkins Job:

- Choose job type: Freestyle Project or Pipeline
- Define triggers: On code push (Git webhook), scheduled (cron), or manual

Step 4: Configure Build Steps:

- Source Code: Add Git repository URL
- Build: Execute Maven goals:

```
mvn clean test
```

- Or use a Jenkinsfile (Pipeline as Code):

```
pipeline {  
    agent any  
    stages {  
        stage('Checkout') { steps { git 'https://github.com/repo.git' } }  
        stage('Test') { steps { sh 'mvn clean test' } }  
    }  
    post {  
        always { publishHTML(target: [reportDir: 'target/surefire-reports']) }  
    }  
}
```

Step 5: Publish Results:

- Use JUnit/TestNG plugin to display results in Jenkins dashboard
- Add HTML Publisher for Extent/Allure reports

Step 6: Additional Considerations:

- Parallel execution: Use Selenium Grid or BrowserStack
- Notifications: Configure email or Slack on test failure
- Environment variables: Store credentials securely in Jenkins

Q154.

[GFG]

What is the purpose of Page Factory in Selenium?

Page Factory is an extension of the Page Object Model (POM) design pattern provided by Selenium's support package. It simplifies and optimizes the initialization of web elements in page object classes.

Key Purposes:

1. Automatic Element Initialization:

- Uses `@FindBy` annotations to declare elements
- `PageFactory.initElements()` initializes all annotated elements automatically
- Eliminates repetitive `findElement()` calls in every method

2. Improved Readability:

- Separates element declaration from test logic
 - Test scripts are cleaner and easier to understand
3. Maintainability:
- If UI changes, only update the locator in the page class – not in every test
4. Reusability:
- Page objects can be shared across multiple test cases
5. Lazy Initialization:
- Elements are located in the DOM only when first accessed (not at class creation)

Example:

```
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    @FindBy(id = "username")
    WebElement usernameField;

    @FindBy(id = "loginBtn")
    WebElement loginButton;

    public LoginPage(WebDriver driver) {
        PageFactory.initElements(driver, this); // Auto-initializes all elements
    }

    public void login(String user, String pass) {
        usernameField.sendKeys(user);
        loginButton.click();
    }
}
```

Q155.

[GFG]

Describe the significance of TestNG in Selenium.

TestNG (Test Next Generation) is a testing framework that greatly enhances Selenium automation:

1. Enhanced Organization and Readability:
 - Annotations (@Test, @BeforeTest, @AfterTest, @BeforeMethod, etc.) clearly define test structure
 - Tests can be grouped by functionality, criticality, or any other criteria
2. Data-Driven Testing:
 - @DataProvider annotation enables parameterized tests with multiple data sets
 - Run the same test with different inputs without duplicating code
3. Parallel Testing:
 - Run tests across multiple threads or machines simultaneously

- Significantly reduces execution time for large test suites
 - Configured in testng.xml: `parallel="methods" thread-count="3"`
4. Dependencies and Sequencing:
- Control test execution order with `@DependsOnMethods`
 - Ensure prerequisites run before dependent tests
5. Detailed Reporting:
- Auto-generates HTML reports with test status, execution time, and stack traces
 - Easier failure analysis and debugging
6. Logging Integration:
- Integrates with Log4j and other logging frameworks
7. Suite Management (testng.xml):
- Configure entire test suites, include/exclude groups, set parameters
8. Flexible and Open-Source:
- Supports multiple languages and integrates with Maven, Jenkins
9. Retry Failed Tests:
- Can automatically retry flaky tests using `IRetryAnalyzer`

Q156.

[GFG]

How do you handle synchronization in Selenium WebDriver?

Synchronization ensures Selenium waits for application responses before proceeding. Types:

1. Implicit Wait (Global):

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

- Tells WebDriver to poll DOM for specified time before throwing `NoSuchElementException`
- Applies globally to all `findElement()` calls
- Easy to set up; may waste time on fast-loading pages

2. Explicit Wait (Condition-based):

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("btn")));
```

- Waits for a specific condition before proceeding
- More efficient and targeted
- Works with conditions like: `elementToBeClickable`, `titleContains`, `alertIsPresent`, etc.

3. Fluent Wait (Customizable):

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofSeconds(5))
```

```
.ignoring(NoSuchElementException.class);
```

- Most flexible; custom polling interval and ignored exceptions
- Best for dynamic content with unpredictable load times

4. Thread.sleep() (Avoid in production):

```
Thread.sleep(3000);
```

- Hard wait – pauses for fixed time regardless of readiness
- Makes tests slow and unreliable
- Only acceptable for quick debugging

Best Practice: Use Explicit Wait or Fluent Wait; avoid mixing implicit + explicit waits.

Q157.

[GFG]

What are the benefits of using Selenium Grid?

Selenium Grid enables distributed and parallel test execution. Benefits:

1. Speed Up Test Execution:

- Run tests on multiple browsers and machines simultaneously
- Significantly reduces overall test suite execution time
- Critical for large regression suites

2. Cross-Browser and Cross-Platform Testing:

- Test on different browsers (Chrome, Firefox, Edge, Safari)
- Test on different operating systems (Windows, Mac, Linux)
- Ensures website works for all users regardless of their environment

3. Efficient Resource Sharing:

- Multiple tests share browser nodes efficiently
- Reduces the number of machines needed

4. Scalability:

- Easily add or remove nodes as testing demands change
- Scale up for major releases, scale down during quiet periods

5. Cost Savings:

- Cloud-based Selenium Grid services reduce need for physical devices
- Pay-per-use models (BrowserStack, Sauce Labs) save hardware costs

6. Better Quality:

- Catching cross-browser issues early prevents production bugs
- Higher confidence in application quality

7. Docker Integration (Selenium 4):

- Run Grid nodes as Docker containers for easy setup and teardown
- Consistent, reproducible test environments

Architecture:

Hub (central server) ← receives test requests

Nodes (browser instances) ← execute the tests

Q158.

[GFG]

What are the benefits of using a Data-Driven Framework in Selenium?

A Data-Driven Framework separates test logic from test data. Benefits:

1. One Script, Multiple Scenarios:

- Write the test once; run it with different data sets
- No need to create separate scripts for each data combination

2. Broader Test Coverage:

- Easily test different input combinations and edge cases
- More bugs discovered with less scripting effort

3. Easy Maintenance:

- When data changes, update only the data file (Excel/CSV/DB)
- Test scripts remain unchanged

4. Readability and Clarity:

- Separating data from logic makes scripts cleaner and easier to understand

5. Collaboration:

- Non-technical testers can manage test data in Excel/CSV without touching code

6. Reusability:

- Shared data sets and test logic can be reused across multiple test cases

How it works:

- Store test data: Excel, CSV, XML, database, JSON
- Read data in test: Apache POI (Excel), OpenCSV, JDBC
- Feed to test: @DataProvider in TestNG

TestNG @DataProvider example:

```
@DataProvider(name = "loginData")
public Object[][] getLoginData() {
    return new Object[][] {
        {"admin", "admin123"},
        {"user1", "pass456"},
        {"guest", "guest789"}
    };
}
```

```
@Test(dataProvider = "loginData")
public void testLogin(String user, String pass) {
    loginPage.login(user, pass);
}
```

Q159.

[GFG]

Explain the concept of Object Repository in Selenium.

An Object Repository is a centralized storage for all web element locators used in automated tests.

Concept:

- Instead of hardcoding locators in every test, store them in a central place
- Tests reference the repository to find elements
- Think of it as a "map" of all UI elements

Types of Object Repositories:

1. Properties File:

```
# locators.properties
username.id=username
password.id=password
loginBtn.xpath=//button[@id='login']
```

2. Excel/CSV File:

Element Name	Locator Type	Locator Value
username	id	username
loginBtn	xpath	//button[@id]

3. Page Object Model (most popular):

```
public class LoginPage {
    @FindBy(id = "username") WebElement usernameField;
    @FindBy(id = "loginBtn") WebElement loginBtn;
}
```

4. XML File:

```
<object name="username" type="id" value="username"/>
```

Benefits:

- Easy Updates – Change locator in one place; all tests update automatically
- Reusability – Same element used across multiple test cases
- Cleaner Scripts – Test logic is separate from UI details
- Better Collaboration – Testers can manage locators without touching test code
- Scalability – Easier to grow and maintain large test suites

Q160.

Discuss the role of the Cucumber Framework in Selenium.

Cucumber is a BDD (Behavior-Driven Development) framework that integrates with Selenium to make tests readable by non-technical stakeholders.

How it works:

1. Gherkin Language (plain English test scenarios):

Feature: User Login

Scenario: Valid Login

Given the user is on the login page

When the user enters username "admin" and password "pass123"

Then the user should see the dashboard

2. Step Definitions (Java code mapping):

```
@Given("the user is on the login page")
public void openLoginPage() {
    driver.get("https://example.com/login");
}
```

```
@When("the user enters username {string} and password {string}")
public void enterCredentials(String user, String pass) {
    driver.findElement(By.id("username")).sendKeys(user);
    driver.findElement(By.id("password")).sendKeys(pass);
    driver.findElement(By.id("loginBtn")).click();
}
```

```
@Then("the user should see the dashboard")
public void verifyDashboard() {
    Assert.assertTrue(driver.getTitle().contains("Dashboard"));
}
```

Benefits:

- Communication Boost – Everyone understands test scenarios (devs, testers, business)
- Living Documentation – Gherkin files serve as always-updated documentation
- Focus on Behavior – Tests describe what the application should do, not how
- BDD/TDD Support – Works with both development approaches
- Reusable Steps – Step definitions can be shared across multiple scenarios

Q161.

What are the considerations for headless browser testing in Selenium?

Headless browser testing runs browsers without a visible GUI (user interface).

Advantages:

1. Faster Execution:

- No rendering of graphics means tests run significantly faster
- Ideal for CI/CD pipelines where speed matters

2. Scalability:

- Run tests on multiple machines or containers without display requirements
- Works well in Docker/cloud environments

3. Resource Efficiency:

- No need for a display server (no Xvfb needed on Linux)
- Lower memory and CPU usage compared to visible browsers

4. Automation-Friendly:

- Can capture screenshots, measure performance, and run background tasks

5. Cost Savings:

- Test mobile/web views without physical devices

Disadvantages / Considerations:

1. Visual Testing Gaps:

- Cannot verify visual layout, animations, hover effects
- Separate visual testing tools needed (Applitools, Percy)

2. Debugging Difficulty:

- Harder to diagnose issues without seeing the browser
- Relies on screenshots and logs

3. JavaScript Compatibility:

- Some complex JS libraries may behave differently in headless mode

4. Browser Compatibility:

- Older browsers may not support headless mode

Setting up headless Chrome:

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless", "--no-sandbox", "--disable-dev-shm-usage");
WebDriver driver = new ChromeDriver(options);
```

Python:

```
options = webdriver.ChromeOptions()
options.add_argument("--headless")
driver = webdriver.Chrome(options=options)
```

Q162.

[GFG]

How do you handle dynamic web tables in Selenium?

Dynamic tables have content that changes through AJAX, pagination, sorting, or filtering.

Challenges:

- No fixed IDs or names – hard to uniquely identify rows/cells
- Content changes between test runs
- Multiple pages of data (pagination)
- Sorting/filtering scrambles element positions

Solutions:

1. Use Relative XPath/CSS Selectors:

```
WebElement cell = driver.findElement(  
By.xpath("//table[@id='dataTable']/tbody/tr[2]/td[3]"));
```

2. Iterate All Rows and Columns:

```
WebElement table = driver.findElement(By.id("myTable"));  
List<WebElement> rows = table.findElements(By.tagName("tr"));  
for (WebElement row : rows) {  
List<WebElement> cells = row.findElements(By.tagName("td"));  
for (WebElement cell : cells) {  
System.out.print(cell.getText() + " | ");  
  
}  
}  
}
```

3. Search for Specific Value:

```
List<WebElement> cells = driver.findElements(By.xpath("//td"));  
for (WebElement cell : cells) {  
if (cell.getText().equals("John")) {  
System.out.println("Found!"); break;  
  
}  
}
```

4. Handle Pagination:

```
while (true) {  
// Process current page rows  
WebElement nextBtn = driver.findElement(By.id("nextPage"));  
if (!nextBtn.isEnabled()) break;  
nextBtn.click();  
  
}
```

5. Wait for Table to Load:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("myTable")));
```

6. Use JavaScript for Direct Extraction:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
```

```
String text = (String) js.executeScript(  
    "return document.querySelector('#table tr:nth-child(2) td:nth-child(1)').innerText");
```

Q163.

[GFG]

Differentiate between WebDriver and WebElement.

WebDriver vs WebElement comparison:

WebDriver:

- Purpose: Controls the browser itself
- Scope: The entire browser window or tab
- Interactions: Opens/closes browsers, navigates to URLs, refreshes pages, manages windows and tabs, handles alerts
- Example: `driver.get("https://www.example.com/")`
- Analogy: Like a remote control for your TV

WebElement:

- Purpose: Represents a specific element within a web page
- Scope: A single element (button, text field, image, link, etc.)
- Interactions: Click, enter text, retrieve text, submit forms, check attributes, check visibility/state
- Example: `element = driver.findElement(By.id("myButton"))`
- Analogy: Like a specific button or channel on your TV

Key Selenium Methods:

WebDriver methods:

- `get(url)`, `navigate()`, `quit()`, `close()`
- `getTitle()`, `getCurrentUrl()`, `getPageSource()`
- `switchTo()`, `manage()`, `findElement()`

WebElement methods:

- `click()`, `sendKeys()`, `clear()`, `submit()`
- `getText()`, `getAttribute()`, `getCssValue()`
- `isDisplayed()`, `isEnabled()`, `isSelected()`
- `getTagName()`, `getRect()`

Relationship:

- WebDriver is used to find WebElements: `driver.findElement(By.id("x"))`
- Once found, you interact with the WebElement directly

Q164.

[GFG]

How do you perform database testing using Selenium?

Selenium WebDriver cannot directly access databases, but you can combine it with database APIs for end-to-end testing.

Approach 1: Action → Verify in DB (most common):

```
// Use Selenium to perform UI action
driver.findElement(By.id("addUser")).click();
// Then verify in database using JDBC
Connection conn = DriverManager.getConnection(dbUrl, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users WHERE name='John'");
if (rs.next()) System.out.println("User saved to DB: " + rs.getString("name"));
```

Approach 2: JDBC (Java Database Connectivity):

```
Class.forName("com.mysql.jdbc.Driver");
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb", "root", "password");
PreparedStatement ps = conn.prepareStatement("SELECT * FROM orders WHERE id=?");
ps.setInt(1, orderId);
ResultSet rs = ps.executeQuery();
```

Python approach:

```
import sqlite3 # or psycopg2 for PostgreSQL, mysql-connector for MySQL
conn = sqlite3.connect("test.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM users WHERE username='admin'")
rows = cursor.fetchall()
```

Approach 3: Mocking the Database:

- Use Mockito to create a mock database for unit-level tests

Approach 4: Testing Frameworks with DB support:

- Robot Framework with DatabaseLibrary
- Cypress with db plugins
- TestNG + JDBC for full integration testing

When to do DB testing with Selenium:

- After form submission – verify data saved correctly
 - Before tests – set up known test data
 - After tests – clean up test data
-